

“Subscription and Billing SaaS for Banking and Fintech”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

VENKATA SAI KRISHNA :2520030027

PUJITH PAVAN KUMAR :2520030029

SRIVARUN :2520030324

Under the Esteemed Guidance of

Dr. S. Spandana

&

Dr. Yerragudipadu Subbarayudu

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled **“Subscription and Billing SaaS for Banking and Fintech”** is a record of Bonafide work of **VENKATA SAI KRISHNA - 2520030027 , PUJITH PAVAN KUMAR - 2520030029 , SRIVARUN - 2520030324** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VENKATA SAI KRISHNA – 2520030027

PUJITH PAVAN KUMAR – 2520030029

SRIVARUN – 2520030324

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **“Subscription and Billing SaaS for Banking and Fintech”** is a bonafide work done and submitted by **VENKATA SAI KRISHNA - 2520030027 , PUJITH PAVAN KUMAR - 2520030029 , SRIVARUN - 2520030324** in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide(s)

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our guides, **Dr. S. Spandana** and **Dr. Yerragudipadu Subbarayudu**, for their patient guidance, constructive criticism and timely suggestions throughout the design, development and documentation of this project. Their insistence on rigour in database design and honest reporting shaped this work.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the Department for Computer Science Engineering for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. We acknowledge the cooperation of all three members of Group 18, and we sincerely thank our parents, friends and classmates for their kind help and cooperation during this work.

VENKATA SAI KRISHNA – 2520030027

PUJITH PAVAN KUMAR – 2520030029

SRIVARUN – 2520030324

Abstract

Banking and fintech organisations increasingly offer subscription-based financial products, but the systems that manage customers, plans, invoices, payments, refunds, loans and repayments are frequently fragmented. Disconnected records, manual billing, difficulty in tracking invoices, payments and overdue loans, and the absence of a centralised view of operations increase cost and risk, and generic software rarely provides the strong tenant separation that financial data demands.

This project presents a multi-tenant Subscription and Billing SaaS platform for banking and fintech companies. Each financial company is a tenant with its own administrators, customers, plans, subscriptions, invoices, payments, loans, reports and support data. Customers and administrators use separate portals to manage subscription plans and benefits, subscriptions, invoices, payments, refunds, loan applications, loan repayments, notifications, support tickets, documents, statements and search. Administrators can review and approve loans, generate repayment schedules, disburse loans, process overdue instalments, reverse repayments and view eight report aggregates with CSV export.

The system is implemented with a React and Vite frontend, a Node.js and Express API gateway that provides a single forwarder with allowed-prefix enforcement, request identifiers, header forwarding and rate limiting, a FastAPI backend using SQLAlchemy, Pydantic and Alembic, and a MySQL 8 database organised in a 26-domain-table scope with keys, constraints and indexes. Security is addressed through hashed passwords, token-based sessions, role-based authorisation, request validation, gateway protections and, above all, company-scoped tenant isolation.

The report distinguishes clearly between implemented, partially implemented and planned capabilities. The frontend is still in preview mode, and OTP delivery, Google OAuth, email transport and an external payment gateway are not yet connected. Event streaming with Kafka, MongoDB activity logging, a Spring Boot reporting service, full-text search, a document storage pipeline, and Docker, Kubernetes, Prometheus and Grafana based deployment are presented as planned architecture, not as delivered features. The work provides a scalable foundation for a unified financial subscription and lending platform and identifies a clear path towards event-driven and cloud capabilities.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives of the Project 1.5 Importance of the Application 1.6 Target Users / Audience	5-12
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	13-15
3	Technology Stack	3.1 Front-End Design 3.2 Back-End Design 3.3 Database Design 3.4 Version Control 3.5 APIs or External Services	16-20
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of Each Layer (Frontend, Backend, Database) 4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines) 4.4 Scalability Considerations	21-24
5	Design	5.2 ER Diagram / Database schema	25-25
6	Implementation	6.1 Module-Wise Implementation 6.2 Front-End Logic (UI Rendering, State Management) 6.3 API Integration 6.4 Authentication and Authorisation	26-41
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	42-48
8	Testing	8.1 Postman Testing (Frontend/Backend) 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	49-52
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	53-55
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	56-58
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimisations	59-60
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned During Development	61-62

S. No.	Chapters	Topics	Page.no
13	References	Books Tutorials and Learning Resources APIs and Tools Used Documentation and Research References	63-64
14	Appendices	Screenshots of the app Sample code snippets Installation/setup instructions User manual or guide Geo Tag photos with guide Review forms with guide signatures	65-69

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Banking and financial technology organisations increasingly deliver their products as subscription-based services. Account packages, premium service tiers, lending products and value-added financial features are now offered to customers on recurring billing terms. Delivering such services requires an organisation to maintain accurate records of its customers, its plans, the subscriptions that link the two, the invoices raised for those subscriptions, the payments that settle the invoices, and any refunds that reverse them. In addition, an institution that also lends money must maintain records of loan products, loan applications, approved loans, repayment schedules and repayments, and must present a consistent view of all this information to both the customer and the administrative staff of the institution.

In practice, these responsibilities are frequently handled by separate systems. Customer records may sit in one application, invoicing in another, loan servicing in a third and support conversations in a fourth. Each system holds a partial view of the same customer, and reconciling those views is a manual and error-prone activity. As the number of customers and products grows, the cost of this fragmentation rises in the form of delayed billing, disputed invoices, unclear loan balances and inconsistent reports. Staff must move between tools to answer a simple question such as how much a particular customer currently owes, and customers must contact the institution because no single screen tells them.

Software-as-a-Service (SaaS) platforms offer a way to address this problem. A single hosted platform can serve many organisations at once, provided that each organisation's data is kept strictly separate. This design is known as multi-tenancy. In a financial setting, multi-tenancy is acceptable only if tenant isolation is treated as a first-class security requirement, because a single cross-tenant data leak would expose the confidential financial records of an unrelated institution. The design of such a platform therefore begins with the question of who owns each record and how that ownership is enforced at every layer of the system.

The project described in this report, “Subscription and Billing SaaS for Banking and Fintech”, is a multi-tenant platform that brings the above concerns into one system. A financial company is modelled as a tenant. Each company has its own administrators, customers, plans, subscriptions, invoices, payments, loans, reports and support data. The platform is implemented with a React frontend, a Node.js and Express API gateway, a FastAPI backend and a MySQL 8 database, and its tagline is “Powering Smarter Financial Services”.

The team also considered, and rejected, the alternative of building separate, narrower applications for billing and for lending, on the grounds that keeping them as two distinct systems would reproduce exactly the fragmentation problem the project set out to solve. Treating subscription billing and lending as two faces of the same customer relationship, sharing one customer record and one company boundary, was judged the more faithful reflection of how a real financial institution actually experiences its customers.

The idea for this project emerged from observing the inefficiencies described above and recognising that a well-structured relational database could unify them. The project was carried out as part of the course DBSE & DBD (Course Code: 25CS1302E) by Group 18 of the Department of Computer Science and Engineering, K L (Deemed

to be) University. The course emphasises database design, software engineering practice and the disciplined development of data-driven applications, and the project applies these ideas to a realistic financial domain in which correctness, traceability and separation of data matter a great deal.

Modern web development frameworks make it practical for a small student team to build such a platform. FastAPI provides typed request validation and automatic API documentation, SQLAlchemy and Alembic provide a maintainable link between Python code and a relational schema, React and Vite provide a responsive component-based interface, and Express provides a compact gateway layer for transport-level protection. Combining these tools with a carefully designed MySQL schema allowed the team to explore, in a single project, the full path from data modelling to a user-facing application.

Throughout this report, the current state of every capability is stated explicitly. Features that are fully built, features that are only partially implemented or stubbed, and features that belong to planned future work are kept clearly separate. The frontend, for example, is substantially built but still operates in preview mode, and several integrations such as real OTP delivery, Google OAuth, email transport and an external payment gateway are not yet connected. This distinction is important for an honest academic account, and it is repeated wherever a capability is discussed.

A useful way to see the scale of the problem is to imagine a single customer who holds both a subscription and a loan with the same institution. In a fragmented system, the customer service representative handling a query about that customer's account would need to open a billing tool to check the subscription, a separate loan servicing tool to check the loan, and possibly a third tool to check any support history, before being able to answer even a simple question. Each additional tool in this chain adds delay and the possibility of overlooking a relevant fact.

1.2 Problem Statement

Conventional banking and fintech subscription and billing arrangements suffer from a set of related problems. Customer information is stored in several places, so a single customer may have different addresses, statuses or balances in different systems. Subscription management is disconnected from billing, which leads to invoices that do not match the plan actually held by the customer. Where billing is manual, invoice preparation is slow and prone to arithmetic and data-entry errors, and tracking which invoices have been paid becomes difficult, particularly when payments arrive through different channels.

Payment tracking and refund handling add further complexity. A payment must be linked to the invoice it settles, and a refund must be linked back to the payment it reverses. When these links are held in spreadsheets or in separate tools, auditing them is laborious, and disputes are difficult to resolve because no single record shows the whole story. Loan management introduces additional difficulty because a loan is not a single event but a lifecycle that includes application, review, approval, scheduling, disbursement, repayment and, in some cases, overdue handling and reversal of mistaken repayments. Repayment tracking and overdue management require consistent arithmetic over principal, interest and penalty components, and small errors accumulate into visible discrepancies.

Reporting is also affected. When data is spread across systems, administrators cannot obtain a single, reliable view of the lending book, the billing summary or the collection performance of the institution. Generic software that is not designed around tenants may additionally provide weak separation between organisations, and scaling such software to many financial companies becomes difficult because each new institution requires a separate deployment. Finally, financial software must satisfy security and audit expectations: access must be controlled by role, requests must be traceable, and data integrity must be enforced at the database level rather than left entirely to the application.

The major problems that motivate this project can therefore be summarised as follows.

1. Fragmented customer records and disconnected subscription management across separate systems.
2. Manual billing, difficult invoice tracking and difficult payment tracking.
3. Complicated refund handling without a clear link between refund, payment and invoice.
4. Loan management complexity, including repayment tracking and overdue management.
5. Lack of centralised reporting and of unified customer and administrator workflows.
6. Weak tenant isolation in generic systems and difficulty in scaling to several financial companies.
7. Security and audit requirements that are difficult to satisfy with loosely connected tools.

A further, more subtle problem is that institutions frequently discover these issues only after they have caused customer-facing harm, such as a customer being billed twice for the same subscription period or a loan repayment being applied to the wrong instalment. Because the underlying systems are not connected, there is no automatic cross-check that would catch such an error before it reaches the customer. Resolving the complaint then requires a member of staff to manually trace the transaction across several tools, which is slow, inconsistent between staff members, and difficult to audit afterwards.

The proposed platform addresses these issues by providing one system in which the relationships between customers, plans, subscriptions, invoices and payments are represented explicitly in a relational schema, in which the loan lifecycle is supported by a dedicated finance module, and in which every record is owned by a company so that tenant isolation can be enforced consistently. It offers customers and administrators separate portals over the same data, produces reports from that data, and protects it with layered security controls. The report does not claim that the current implementation is a production financial system; it presents a working academic implementation whose design anticipates production concerns.

These problems are not merely operational inconveniences; they carry real financial and reputational risk. A billing error that goes uncorrected can lead to customer churn, and a loan portfolio that cannot be summarised accurately can lead to poor lending decisions. Addressing the problem therefore has both a technical dimension,

namely building a system that keeps records consistent, and an organisational dimension, namely giving staff a single place to look for the truth about a customer's account.

It is also worth noting what this project deliberately does not attempt. It does not attempt to replace an institution's core banking system, nor does it attempt to handle regulatory reporting to a central bank or similar authority. It is scoped as a subscription and lending front end and administrative layer that sits above such systems in a real deployment, focused on the customer- and administrator-facing workflows described in this report.

1.3 Scope of the Project

The scope of the project covers company (tenant) management, administrators, customers, authentication and sessions, subscription plans and plan benefits, subscriptions, invoices, payments, refunds, transactions, loan products, loan applications, loans, repayment schedules, repayments, notifications, support tickets and support messages, documents, customer statements, administrative reports, global search and audit or event tracking. The database scope comprises 26 domain tables in MySQL 8, of which the principal entities are companies, admins, customers, otp_verifications, plans, plan_benefits, subscriptions, loan_applications, loans, loan_terms, repayments, invoices, payments, refunds, transactions, notifications, support_tickets, support_messages and documents.

Because several capabilities are still under development, each is classified by its implementation status, and this classification is used consistently in the rest of the report. The following components are implemented in the current project: the substantially built React user interface with routes, a design system, theming and accessibility work; the Node.js and Express gateway, which is complete for the current scope; the FastAPI backend with customer and administrator APIs for the implemented resources; the MySQL database with SQLAlchemy models and Alembic migrations; the finance module for schedules, disbursement, repayment, overdue marking and reversal; eight administrative report aggregates with CSV export; customer statements; notifications; and a SQL LIKE based global search.

The following components are partially implemented or stubbed. The frontend still operates in preview mode, and live API wiring has not yet been fully switched on. Authentication and session infrastructure with JSON Web Tokens exists, but one-time-password delivery is not connected to a real service, Google OAuth is not connected to a production provider flow, real email transport is not enabled, and development or stubbed authentication is still used in places. The document and support ticket domain and API exist within the implemented scope, but a complete file-upload storage pipeline does not. The CompanyInvite model exists, but the full invite and activity log interface and API are not yet complete.

The following components are outside the current scope and are treated only as planned architecture: Kafka event streaming, a Spring Boot reporting service, MongoDB activity, usage and event logging, a real external payment gateway, production Google OAuth, production email and OTP delivery, full-text search, a complete document upload and storage pipeline, and production deployment on Kubernetes. Docker, Docker Compose and Kubernetes form the planned deployment architecture, and Prometheus and Grafana form the planned monitoring architecture. None of these is described in this report as already implemented.

The scope was deliberately bounded so that the team could build a working system within the time available for the course, rather than attempting every feature of a commercial platform at a shallow level. Wherever a choice had to be made between breadth and depth, the team preferred to implement a smaller set of capabilities completely, with real validation, real constraints and real tests, over implementing a larger set only superficially. This is why, for example, the loan lifecycle is implemented in full even though the payment gateway behind it is simulated: the team judged that the internal correctness of the lifecycle was the more valuable thing to demonstrate within an academic project.

An additional, more informal objective guided day-to-day decisions throughout the project: whenever the team was unsure whether a feature belonged in the current phase or in future work, the question asked was whether omitting it would leave the core billing or lending flow incomplete, or whether it was an enhancement layered on top of an already-complete flow. Features in the first category were prioritised; features in the second category, such as full-text search or advanced analytics, were deferred to Chapter 11.

1.4 Objectives of the Project

The main objective of the project is to design and develop a unified, multi-tenant subscription and billing platform for banking and fintech companies, with a relational database at its centre and with clearly documented security and integrity measures. The specific objectives are as follows.

1. To design a multi-tenant relational database that represents companies, customers, plans, subscriptions, invoices, payments, refunds, transactions and loans with enforced integrity.
2. To implement a FastAPI backend that exposes customer and administrator functionality with validated inputs and role-based access.
3. To implement an Express-based API gateway that forwards requests safely, applies rate limiting and propagates request identifiers and tenant context.
4. To build a React user interface for customers and administrators with reusable components, a design system and accessibility considerations.
5. To implement the loan lifecycle, including schedule generation, disbursement, repayment, overdue marking and repayment reversal, using careful monetary arithmetic.
6. To provide administrative reporting, customer statements and tenant-scoped global search.
7. To document the system honestly, distinguishing implemented, partially implemented and planned capabilities.

Underlying these objectives is a wider academic aim: to apply the principles taught in the course, namely normalisation, keys and constraints, transaction integrity, indexing and migration management, to a system that is large enough to expose real design trade-offs. The project also aims to give each team member experience of a

different layer of the stack, so that the final result reflects collaboration between database design, backend development and frontend development.

These objectives were written at the start of the project and were used throughout development to decide what to build next. When a proposed feature did not clearly serve one of these objectives, it was deferred to the future enhancements described in Chapter 11 rather than built immediately, which helped the team keep the project within scope for the course timeline while still producing a coherent, working system rather than a collection of unrelated demonstrations.

The project also has value simply as a demonstration that a small student team can design and build a system of this size within a single course, provided that the scope is managed deliberately. Many of the design decisions described in later chapters, such as separating configuration from transactional history, or keeping the gateway free of business logic, are patterns that generalise well beyond this specific project and are transferable to other systems the team members may build in future.

1.5 Importance of the Application

The importance of the application lies in its treatment of billing and lending as related activities within one data model. For a financial company, the ability to trace an invoice to a subscription, a payment to an invoice, or a repayment to a loan schedule reduces reconciliation effort and supports audit. For customers, a single portal that shows subscriptions, invoices, payments, loans, notifications and statements removes the need to consult several systems and gives them a clear picture of what they owe and what they have paid.

The multi-tenant design is important because it allows a single platform to serve several financial companies without mixing their data. Each company retains ownership of its plans, customers and records, while sharing the cost of the underlying software. Tenant isolation is therefore not an optional feature but the property that makes the shared design acceptable, and much of the design effort in this project was directed towards ensuring that it holds at every layer.

From an engineering perspective, the project demonstrates how database constraints, layered service design and a defensive gateway can be combined in a multi-tenant application. It also demonstrates disciplined handling of monetary values, which is a recurring source of defects in financial software, and it shows how migrations can be used to evolve a schema safely, as in the migration that added the loan interest columns. Finally, as an academic project it provides a substantial case study in relational modelling, API design, migration management and testing that future batches can extend.

The importance of the project is not limited to its technical content. It also lies in the discipline of stating clearly what has been built and what has not, which is a skill that is easy to underestimate in academic work but is essential in professional software development. A system that is described accurately, including its gaps, is more useful to the people who must decide whether to extend it than a system that is described only in terms of its intentions.

It is worth stating plainly who is not yet a user of the platform in its current state: real paying customers of a real financial institution. Because several integrations remain stubbed, as described throughout this report, the platform

in its present form is suited to demonstration, evaluation and further development, and the report does not present it as ready for that final audience.

1.6 Target Users / Audience

The first group of target users consists of customers of a financial company. These users select a company and a plan, subscribe, view invoices and payments, request refunds, apply for loans and repay them, obtain statements, receive notifications, raise support tickets and search for their own records. Their interest is in clarity and convenience: they want to know at any moment what they have subscribed to, what they owe and what has been paid.

The second group consists of company administrators. They configure plans and benefits and loan products, manage customers with the help of a consolidated Customer 360 view, review and approve loan applications, generate repayment schedules, disburse loans, process overdue instalments, reverse mistaken repayments, decide refund requests, handle support tickets and document records, and view reports. Their interest is in control and visibility over the operations of their own company.

The financial companies themselves are the tenants of the platform, and the platform is intended to let them operate subscription and lending services on shared infrastructure with isolated data. Course faculty and evaluators form an academic audience who will assess the quality of the database design, the architecture, the implementation and the honesty of the status reporting. Finally, future developers form a technical audience who may extend the platform with planned components such as event streaming, a real payment gateway and cloud deployment.

These groups of users are not entirely separate in practice. A member of the development team, for example, may also act as an administrator while demonstrating the system, and a course evaluator may examine the platform from the perspective of a customer in order to judge its usability. The design of the interface and the API was kept simple enough that a person unfamiliar with the system could explore it in either role without extensive instruction, which was itself treated as a design goal alongside the functional requirements listed earlier in this chapter.

Data privacy was a further consideration in scoping the project. Because customer and loan data is sensitive by nature, the team treated the separation between companies not only as a technical requirement but as an ethical one, and the same principle guided decisions such as never logging a customer's password or token in plain text, and never returning more information in an API response than the specific screen making the request actually needed.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The platform was developed and exercised on ordinary student development machines, and no specialised hardware is required to run it. For a comfortable development experience with the frontend development server, the gateway, the backend and a MySQL server running together on a single computer, a processor with at least two cores is recommended, and a quad-core processor is preferable when a container runtime is also in use. The recommendations in this section are practical guidelines drawn from the team's own experience and have not been formally benchmarked.

Memory is the resource that most affects the development experience. A minimum of 8 GB of RAM is recommended because the Vite development server, the Node.js gateway, the Python backend, the MySQL server, the editor and a web browser all run at the same time. A machine with 16 GB is preferable when Docker is used alongside the services, because containers add their own overhead. Storage requirements are modest: about 10 GB of free disk space is sufficient for the dependency folders, the database files, the repositories and the tools.

A display of at least 1366×768 pixels is sufficient for development, while a full HD display is preferable for reviewing the responsive user interface and the diagrams. A broadband network connection is needed for installing packages and for consulting documentation. Because the interface is responsive, it can also be reviewed on a phone or tablet connected to the same local network, although this was used only informally.

A production deployment would have different requirements. It would call for separately sized application servers, a managed or clustered MySQL service with backups, and network security controls such as firewalls and TLS termination. Such sizing has not been performed and is outside the scope of the current work, and the figures above should not be read as production sizing guidance.

The hardware requirements above assume that the reader is setting up the project for the first time on a machine that is also used for other coursework. In practice, the team found that closing unrelated applications while running the full stack, particularly other Node.js or Python processes, made a noticeable difference to responsiveness, since the frontend build tool, the gateway and the backend each start their own process and consume memory independently.

2.2 Software Requirements

The software required to run the platform is entirely open source or freely available for academic use. The operating system may be Windows, Linux or macOS, because all of the tools used are cross-platform. Python 3 is required for the backend, together with the packages listed in the backend requirements, including FastAPI, Uvicorn, SQLAlchemy, Alembic, Pydantic and pytest. A virtual environment is recommended so that the project's dependencies do not interfere with other Python projects on the same machine.

Node.js, in a current long-term-support release, is required both for the Vite tooling of the React frontend and for the Express gateway. The npm package manager installs the JavaScript dependencies of each. MySQL Server 8

is required for the relational database, and the database is created under a name chosen for the project. A modern web browser such as a Chromium-based browser or Firefox is needed to run and review the frontend.

Git and a GitHub account are required for version control and collaboration. Docker and Docker Compose are optional at present, since containerisation belongs to the planned deployment architecture rather than to the current running system. Kubernetes, Prometheus and Grafana are likewise planned components and are not required to run the project today. Exact package versions should be taken from the requirements and package files of the repository rather than from this report, because those files are the authoritative record.

Version compatibility between these tools mattered more than might be expected. Because SQLAlchemy, Alembic and FastAPI evolve together, the team kept the versions of these three packages pinned in the backend requirements file rather than allowing them to update independently, which avoided a class of subtle errors in which a newer version of one package changed behaviour that another package depended upon. The same discipline was applied to the Node.js dependencies of the gateway and the frontend.

The team also kept a short shared document listing the exact versions of Python, Node.js and MySQL used by each member, since a mismatch in major versions occasionally produced subtly different behaviour, particularly in how MySQL handled certain constraint checks between versions. Recording these versions in one place made it faster to rule out an environment difference when a defect appeared to reproduce on one member's machine but not another's.

2.3 Development Tools and Frameworks

Visual Studio Code was the principal editor used by the team for Python, JavaScript and JSX files. Its extensions for Python, ESLint and Git integration made it practical to work on the backend, the gateway and the frontend from a single workspace. MySQL Workbench was used to inspect the schema, run queries and confirm the effect of migrations, and it proved valuable for checking that constraints behaved as designed. Postman was used as an API client for manual testing of the gateway and the backend, allowing requests to be composed with the correct headers and tokens.

Git and GitHub were used for source control and collaboration. Branches allowed team members to work on the database, backend and frontend in parallel, and pull requests provided a point at which changes could be reviewed and merged. Figma was used for user interface design before implementation, which allowed layouts, colours, spacing and component variants to be agreed by the team in advance and reduced rework when the interface was implemented in React.

The frameworks used in the implementation are React, Vite, React Router and Axios on the frontend; Express.js, Helmet, CORS and express-rate-limit on the gateway; and FastAPI, Pydantic, SQLAlchemy, Alembic and Uvicorn on the backend, with pytest for automated tests. Each of these was chosen for a specific reason, which is explained in Chapter 3. The tools and frameworks together allowed a small team to build a layered application in which each layer could be developed and tested on its own.

Beyond the individual tools, the team adopted a small number of conventions to keep the three parts of the system consistent: a shared naming convention for endpoints, a shared approach to error responses, and a shared expectation that every new table would be accompanied by an Alembic migration rather than a manual schema change. These conventions are not visible in any single tool but were as important to the smooth running of the project as the tools themselves.

The team also weighed the cost of learning three separate technology stacks against the benefit of using the most suitable tool for each layer. Using a single stack throughout, for example Node.js everywhere, would have reduced the learning burden but would have made the gateway and the backend less clearly separated in practice, since it is easier to accidentally let business logic creep into a gateway written in the same language and style as the backend behind it. The team judged the extra learning cost worthwhile for the clarity it produced.

Choosing tools that are widely documented and widely used in industry, rather than newer or less common alternatives, was also a deliberate decision. It meant that when the team encountered an unfamiliar error message, a solution or explanation was usually already available in the official documentation or in the wider community, which kept the pace of development steady even when a team member was working with a particular tool for the first time.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend is a single-page application built with React and bundled with Vite. Vite provides a fast development server based on native ES modules and an optimised production build, while React supplies the component model that allows the interface to be composed of small, reusable parts. The interface is written in JavaScript with JSX. React Router defines the customer and administrator routes so that each audience sees only the screens relevant to it, and Axios is used as the HTTP client for communication with the gateway.

The user interface was designed in Figma before implementation. Working in Figma allowed the team to agree on layouts, colour choices, typography and component variants in advance, and to review the flow of screens such as the customer dashboard, the invoice list and the loan application form before writing code. In code, the interface is composed of reusable components such as buttons, form fields, cards and data lists, and is governed by a design system that centralises theming values such as colours, spacing and typography, so that a change to a theme value is reflected consistently across the application.

Accessibility considerations were included in the design work. These include semantic structure, keyboard navigation, visible focus states and sufficient colour contrast, and they are handled in the shared components so that they apply consistently to every screen. The layout is responsive across common screen sizes, so that the same interface can be used on a desktop browser and on a smaller device without a separate code base.

At the time of writing, the user interface and its routes are substantially built, but the frontend continues to operate in preview mode. In this mode the interface presents representative data rather than depending on the live backend for every screen. Wiring the interface fully to the live API has not yet been completed, and this is recorded as a limitation in Chapter 10. The work that remains consists of replacing preview data sources with Axios calls through the gateway, adding loading and error states, and handling token storage.

The choice to build the frontend as a single-page application, rather than as a set of separately rendered pages, was made because the customer and administrator experiences both involve moving between related views, such as from a subscription to its invoices and from an invoice to its payments, without a full page reload. React's component model made it straightforward to keep these related views in sync, since a change in one part of the state can be reflected in every component that reads it.

The interface was also built with a particular attention to how a financial figure is displayed, since a customer reading their invoice or their loan balance needs to trust the number shown to them. Currency values are formatted consistently across every screen that displays them, using a single shared formatting function in the design system, so that the same amount is never shown with a different number of decimal places or a different symbol on two different screens.

3.2 Back-End Design

The backend is implemented in Python with FastAPI, a modern web framework built on the Starlette toolkit and Pydantic. FastAPI was chosen because it combines high performance with type-driven validation and automatic generation of an OpenAPI description of the API, which the project uses for API, import and specification checks. Uvicorn serves as the ASGI server that runs the application. The backend exposes customer and administrator resources through routers, and each route declares the shape of its request and response.

Pydantic models declare the shape and constraints of request and response payloads, so that malformed input is rejected before it reaches business logic. This validation is the first line of defence against invalid data, for example a negative amount or a missing field. Business rules are implemented in service functions rather than in the route handlers, so that handlers remain short and the rules can be tested in isolation. A separate finance module contains the monetary arithmetic used for repayment schedules, repayment allocation, disbursement, overdue marking and reversal.

Persistence is handled with SQLAlchemy, an object-relational mapper that represents tables as Python classes and expresses queries in a composable form. Alembic manages schema evolution through versioned migration scripts, so that any environment can be brought to the same schema by applying the migration history. The current migration revision is 563b915e37ff, which added the loan columns `total_interest_payable` and `interest_discount_applied_pct` and was generated and applied successfully.

Authentication dependencies are applied to protected routes so that the caller's identity, role and company are available to every handler. This design keeps authorisation logic out of individual handlers and makes tenant scoping a property of the framework wiring rather than something each developer must remember. Automated tests are written with pytest, and Python compile checks and API, import and OpenAPI checks confirm that the application is importable and that its specification can be generated.

An alternative considered by the team was to implement the backend in Node.js, using the same language as the gateway and the frontend. FastAPI was preferred instead because of its strong integration with Pydantic, which gave the team declarative, type-checked validation with comparatively little code, and because Python's ecosystem for numerical and financial calculations, together with pytest, suited the finance module well.

Field names in the schema were chosen to be self-explanatory wherever possible, for example `total_interest_payable` and `interest_discount_applied_pct` rather than shorter, less descriptive abbreviations, on the view that a schema that can be understood by reading its column names reduces the need to consult separate documentation when working with the database directly, for instance in MySQL Workbench.

3.3 Database Design

The database is MySQL 8, and it is the centre of the design. The design is relational and covers a 26-domain-table scope. Primary keys identify each row, foreign keys express ownership and lifecycle relationships where applicable, unique constraints prevent duplicate business identifiers, check constraints restrict values to valid ranges, and indexes support frequent lookups. The principal entities are companies, admins, customers,

otp_verifications, plans, plan_benefits, subscriptions, loan_applications, loans, loan_terms, repayments, invoices, payments, refunds, transactions, notifications, support_tickets, support_messages and documents, and the remaining tables support loan products, repayment schedules, invoice items, company invites and authentication sessions and event tracking.

Tenant or company ownership is a central design principle. Records that belong to a company are associated with it, either directly or through a parent record that has such an association, so that queries can be scoped to that company. A customer, for example, belongs to a company, a subscription belongs to a customer and a plan, and an invoice belongs to a subscription. Following these relationships always leads back to a company, which is what makes it possible to guarantee that a user of one company can never reach the records of another.

Financial data integrity is treated as a design goal rather than an afterthought. Relationships such as invoice to payment or loan to repayment are modelled explicitly, and constraints are used so that invalid states are rejected at the database layer as well as in application code. The two loan columns added in the most recent migration record the total interest payable on a loan and the percentage interest discount applied to it, so that the financial position of a loan can be explained from stored values rather than recomputed from assumptions.

Schema changes are made only through Alembic migrations generated from the SQLAlchemy models. This discipline means that the models, the migrations and the database cannot silently drift apart. MySQL Workbench was used to inspect the database after migrations and to confirm that keys, constraints and indexes had been created as intended. The entity-relationship diagram of the database is presented in Chapter 5.

The choice of MySQL over a document-oriented database such as MongoDB was deliberate. The relationships between companies, customers, plans, subscriptions, invoices, payments and loans are naturally tabular and benefit from foreign keys and constraints that a relational database enforces automatically. A document database was considered only for the planned activity-log component described in Chapter 11, where the data is less structured and where the relational guarantees used elsewhere in the platform are less important.

The backend's reliance on typed schemas also served the team well during onboarding. When a new team member needed to understand what a particular endpoint expected or returned, the Pydantic schema and the OpenAPI description generated from it were usually sufficient, without needing to read through the implementation of the route itself, which reduced the amount of tacit knowledge that existed only in one team member's head.

3.4 Version Control

Git was used as the version control system for the entire project, and GitHub was used to host the shared repository. Every change to the source code, from the first database models to the latest frontend components, was recorded as a commit with a message describing its purpose. This history provides a record of how the project evolved and allows any earlier state of the system to be recovered if a change introduces a problem.

The team used branches so that database, backend and frontend work could proceed in parallel without interfering with one another. A team member working on a new API endpoint, for example, could do so on a separate branch while another member refined a screen, and the changes were merged once they had been reviewed. Pull requests

on GitHub provided a natural point for discussion, and merge conflicts, which were occasionally unavoidable when two members touched the same file, were resolved by agreement between the members concerned.

Sensitive configuration was kept out of version control. Database credentials, the secret used to sign tokens and similar values were supplied through environment variables, and files that hold them were excluded from the repository. This practice prevents secrets from being exposed through the repository history and mirrors what would be expected in a production environment. Documentation, including this report's diagrams and setup notes, was also kept alongside the code so that it could be versioned in the same way.

Commit messages followed a simple convention describing the module affected and the nature of the change, for example a prefix identifying whether the commit touched the database, the backend, the gateway or the frontend. This made it easier to locate, weeks later, the commit that introduced a particular table or endpoint, and it made the project's history a more useful record than a sequence of generic messages would have been.

.gitignore rules were used from the start of the project to exclude environment files, build artefacts and dependency folders from the repository, which kept the repository focused on source code and documentation rather than on generated or machine-specific files. This is a small discipline, but the team found that establishing it early avoided the more disruptive task of removing such files from the repository's history later in the project.

A short internal convention document was maintained describing how new endpoints should be named and structured, so that a new resource added late in the project, such as the loan interest fields, would follow the same conventions as resources added at the very start, keeping the API consistent even as it grew from a handful of endpoints to the full set described in Chapter 6.

3.5 APIs or External Services

The platform exposes its functionality through a REST-style API implemented in FastAPI and reached through an API gateway implemented with Node.js and Express. The gateway follows a single-forwarder architecture: all permitted requests are proxied through one forwarding mechanism to the FastAPI backend using Axios. Only requests whose paths begin with an allowed prefix are forwarded, and other paths are refused. The gateway forwards the request identifier, the Authorization header and the X-Tenant-ID header, and it handles both JSON and CSV or text payloads so that report exports can pass through unchanged. Unsupported methods receive a 405 response.

Helmet applies standard HTTP security headers, CORS restricts the origins that may call the gateway, and express-rate-limit throttles clients. The rate-limit configuration includes IPv6-safe key handling so that clients on IPv6 networks are not able to bypass limits by varying the low-order address bits. The gateway is complete for the current scope. JSON Web Tokens are used to carry the identity, role and company of an authenticated user, and passwords are stored as hashes rather than as plain text.

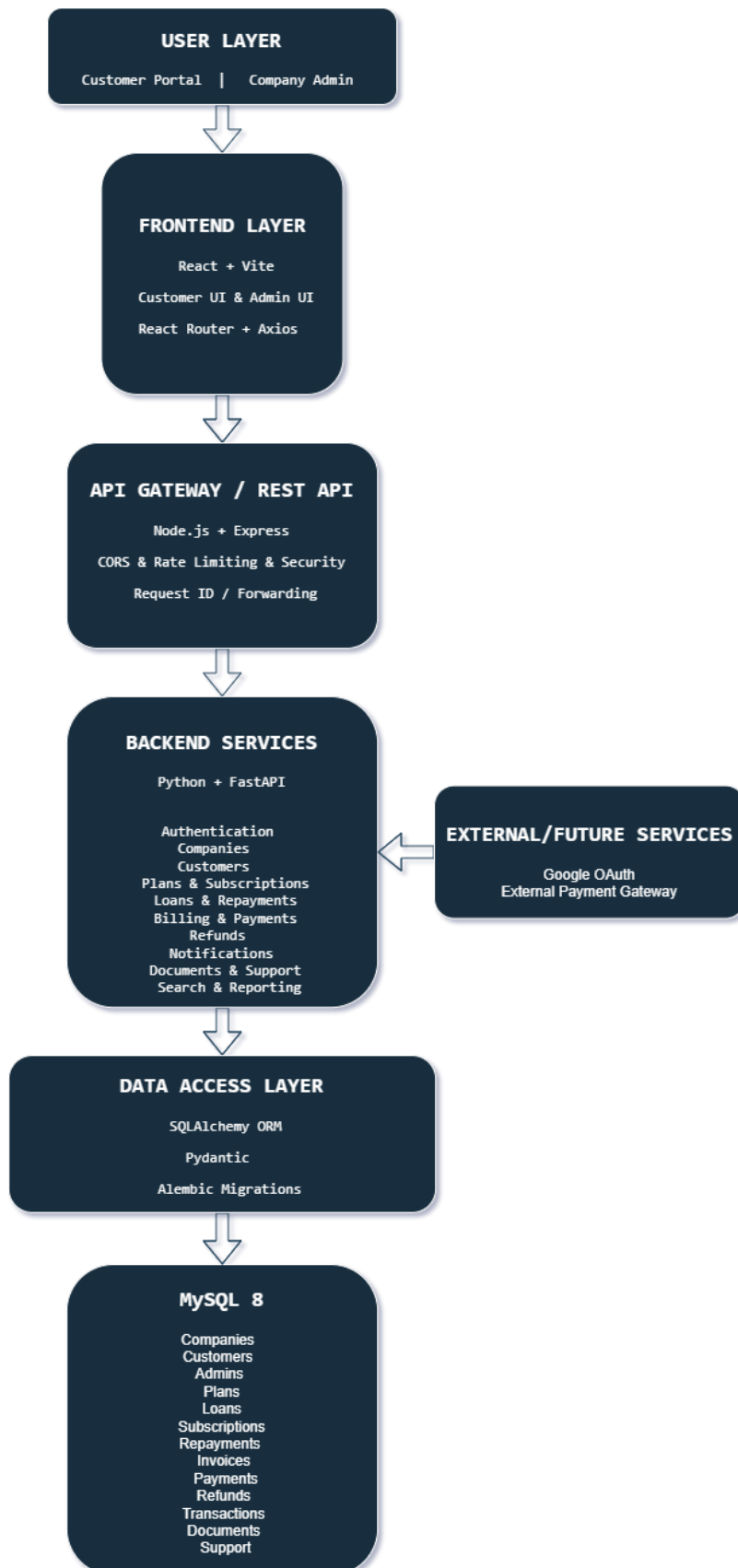
No external service is currently connected to the running system. Payments are simulated within the platform, one-time-password delivery and email transport are stubbed, and Google OAuth is not connected to a production provider flow. These integrations, together with Kafka event streaming, MongoDB event logging and a Spring

Boot reporting service, are described in Chapter 11 as planned enhancements. Documentation and tooling services that were used during development include the official documentation of the frameworks, Postman for API testing, and Figma for interface design. The report makes no claim of compliance with PCI DSS, ISO 27001, SOC 2 or any regulatory certification.

Finally, the team recognises that no external service was integrated during development, which kept the project fully runnable offline and made it straightforward for any team member to set up a working copy without needing credentials for a third-party account. This is convenient for an academic project but is explicitly one of the limitations that would need to be addressed, as described in Chapter 10 and Chapter 11, before the platform could serve real customers.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



The diagram distinguishes between components that are running today and components that are labelled PLANNED. This distinction is maintained visually as well as in the surrounding text, because a diagram that mixed implemented and planned components without a clear label would risk giving an evaluator the impression that the platform is more complete than it currently is, which is precisely the kind of overstatement this report tries to avoid throughout.

4.2 Description of Each Layer (Frontend, Backend, Database)

The architecture of the platform follows a layered approach in which each layer has a single responsibility. This separation enhances modularity and maintainability and allows each layer to be developed and tested independently while the whole system still works smoothly together. The first layer, the frontend, is the visual and interactive face of the platform. It is built with React and Vite, uses React Router to separate customer screens from administrator screens, and uses Axios to call the gateway. Reusable components and the design system keep the interface consistent. In the current state the layer operates in preview mode, and the live API integration is the principal remaining task for this layer.

The second layer is the API gateway. It is the only network entry point exposed to the frontend. It validates that the requested path begins with an allowed prefix, applies Helmet headers and CORS policy, enforces rate limits, attaches or forwards a request identifier, forwards the Authorization and X-Tenant-ID headers, and relays the response. By concentrating these concerns in one place, the backend remains simpler and the behaviour of every route is uniform. The gateway does not contain business logic; it neither interprets financial data nor makes authorisation decisions, which remain the responsibility of the backend.

The third layer is the FastAPI backend. It contains routers for customer and administrator resources, Pydantic schemas for validation, service functions that implement business rules, a finance module for monetary calculations, and SQLAlchemy models for persistence. The backend is stateless with respect to requests: each request carries the token that identifies the caller, and no request depends on in-memory state left by an earlier request. This property allows additional backend instances to be added in the future without changing the code.

The fourth layer is the database. It is a MySQL 8 instance whose schema is defined by the SQLAlchemy models and evolved through Alembic migrations. The database is the final authority on data integrity: even if a defect in application code attempted to store an invalid value, a check constraint, a unique constraint or a foreign key would reject it. Indexes support the lookups that occur most frequently, such as retrieving the invoices of a customer or the instalments of a loan.

Two concerns cut across the layers. Security is provided jointly by the gateway, which contributes rate limiting, header hardening and request identifiers, and by the backend, which contributes password hashing, token and session handling, role-based authorisation, tenant resolution and validation. Reporting is implemented inside the backend as aggregate queries over company-scoped data, with eight administrative report aggregates and CSV export. A separate Spring Boot reporting service has been considered as future architecture but is not part of the current implementation.

A typical authenticated request illustrates how the layers cooperate. The browser sends the request to the gateway, which applies Helmet headers and CORS policy, checks the rate limit and verifies the path. The gateway attaches a request identifier and forwards the request, together with the Authorization and X-Tenant-ID headers, to the backend. In the backend, a dependency validates the token and resolves the user, the role and the company. The router validates the payload, and a service function applies business rules and queries the database with the company scope. The response travels back through the gateway to the client, and the request identifier allows the whole exchange to be correlated.

Tenant isolation is the most important architectural property. A financial company is a tenant, and a user belonging to one company must not be able to access another company's records. The design enforces this by resolving the company from the authenticated identity and applying a company filter to data access, with the gateway additionally forwarding the X-Tenant-ID header so that tenant context is available end to end. Isolation matters especially in banking and fintech because the data involved includes identity details, balances, repayment behaviour and payment history, and a leak between tenants would breach the confidentiality owed to each institution's customers.

An alternative architecture considered early in the project was to let the frontend call the FastAPI backend directly, without a gateway in between. This was rejected because it would have required every cross-cutting concern, rate limiting, header hardening, origin control and request identification, to be implemented inside the backend itself, and because it would have coupled the frontend to backend-specific address changes. The gateway, in exchange for one additional network hop, keeps these concerns in a single, small codebase that is easier to reason about and to test.

Error handling was designed to cross the layers consistently. A validation failure in the backend produces a structured error response, which the gateway relays unchanged, and which the frontend is designed to translate into a message next to the relevant form field. A network-level failure, such as the backend being unreachable, is handled separately by the frontend's data service layer, which distinguishes a business-rule rejection from a connectivity problem so that the message shown to the user is appropriate to the actual cause.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The current deployment is a local development arrangement in which the frontend development server, the gateway, the backend and MySQL run on the developer's machine. This arrangement is sufficient for development and demonstration. The frontend runs on the Vite development server, the gateway and the backend run as separate local processes, and MySQL runs as a local server hosting the project database. No cloud deployment has been performed, and no continuous integration or continuous deployment pipeline has been configured.

The planned architecture places each component in a container: one for the frontend static build, one for the gateway, one for the backend and one for MySQL. Docker Compose is planned to orchestrate the containers during development, with a private network, a persistent volume for the database and environment configuration, so that a developer can start the whole system with a single command. For production, Kubernetes is the planned target. Each service would run as a Deployment with several replicas behind a Service, external traffic would

enter through an Ingress, configuration would be held in ConfigMaps, secrets would be held in Secrets, and horizontal autoscaling would adjust the number of replicas to the load. Prometheus and Grafana would collect and display metrics.

A continuous integration pipeline is also a natural future addition. On each push to the repository, the pipeline could run the Python compile checks, the pytest suite and the frontend build, so that regressions are detected early, and it could build container images for deployment. None of this has been implemented, and it is described here only to show how the present development arrangement is intended to evolve. The distinction between the current development deployment and the future production architecture is maintained throughout the report.

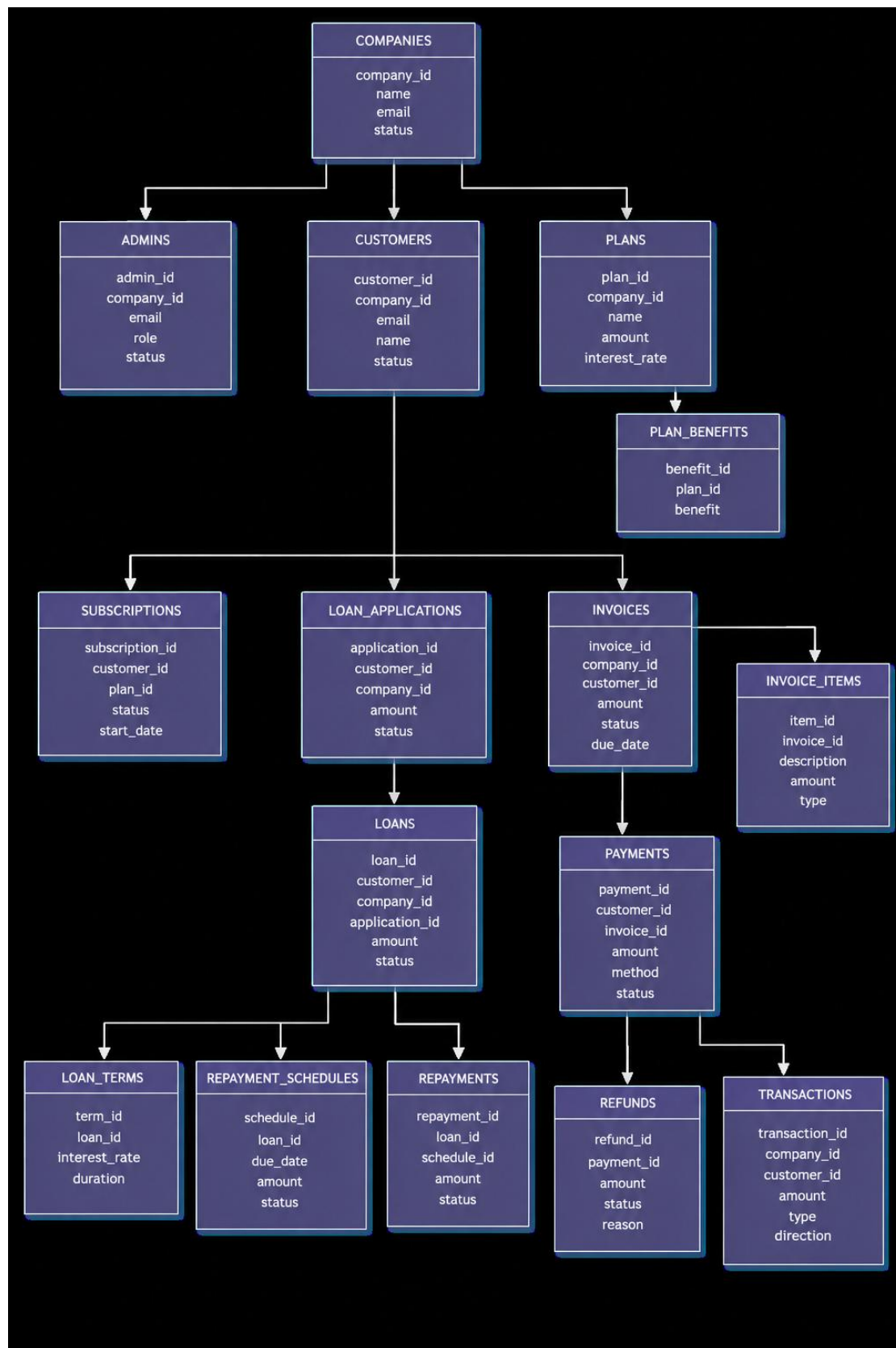
4.4 Scalability Considerations

Although the current deployment is a single machine running every component, the layered architecture was designed with future scaling in mind. Because the backend does not keep request-specific state in memory, more than one backend instance could be run behind the gateway once the gateway is extended to distribute requests across them. The database is the component most likely to become a bottleneck as data grows, and the schema's indexes were chosen with the most frequent queries, such as retrieving a customer's invoices or a loan's schedule, specifically in mind.

Scalability was treated as a property to design for rather than a property to build immediately, which is consistent with the academic scope of the project. The team's judgement was that demonstrating correct, tenant-isolated behaviour on a modest dataset was of greater value for this course than prematurely optimising for a scale of data that the project does not yet have, and the planned Kubernetes architecture described in Chapter 9 is where such scaling would eventually be exercised.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database schema



CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

The implementation of the platform was structured into interrelated modules that together form a cohesive system. Each module was designed with a clear functional boundary while maintaining strong interoperability through a shared data architecture and a unified interface logic. This modular structure ensures scalability and maintainability and allows each component to evolve independently. The platform comprises twenty-nine modules, which are described in this section in the order in which a customer or an administrator would meet them. For every module the description covers its purpose, the user perspective, the technical implementation, the data involved, the role of the module in the API, the security considerations, the workflow and the current implementation status.

Module 1: Company/Tenant Management Company/Tenant Management is the foundation of the multi-tenant design. Its purpose is to represent each financial company as an isolated tenant that owns its administrators, customers, plans, subscriptions, invoices, payments, loans, reports and support data. From the user's perspective the company is fixed by sign-in, so that users simply work within their own company. Technically, the company entity is the ownership root that other entities reference, and the company is resolved from the authenticated identity rather than from client-supplied data. Every query that touches tenant data is scoped by company, which is the main security consideration of the module. The module is implemented.

Because the company entity sits at the root of the ownership graph, its design received particular scrutiny during the database design phase. Every other tenant-scoped table either references companies directly or references a table that does, and this property was checked deliberately for each new table added to the schema, so that no table could accidentally become reachable without a company filter.

In practice, the company that a user belongs to is set once, at the point the account is created, and does not change afterwards in the current implementation. This simplification avoided the additional complexity of handling a customer or administrator who might need to move between companies, which was judged unlikely to occur often enough within the scope of this project to justify the extra design effort.

A future enhancement recorded in Chapter 11 would allow a super-administrator role that can see summary figures across all companies for platform-level monitoring purposes, without weakening the isolation that applies to every other role; this was deliberately left out of the current phase to avoid introducing an exception to the isolation rule before the rule itself had been thoroughly tested.

Module 2: Admin Management Admin Management provides the company staff who configure and operate the platform. Administrators sign in to a separate administrator interface, and their records are linked to a company. Administrative routes are protected by role checks so that customer tokens are refused, and every administrative action is limited to the administrator's own company. The module is implemented, although the complete company invite and activity log interface and API are not yet finished; the CompanyInvite model exists and the remaining scope is recorded as a limitation.

The decision to give administrators a separate sign-in area, rather than a single login screen for every user, reflects the different sets of actions each role can take. It also simplifies the frontend, because the administrator area can assume a signed-in administrator throughout and does not need to branch its navigation based on role.

Administrator accounts are created by another administrator of the same company rather than through open self-registration, which reflects the expectation that administrator access is granted deliberately by an institution rather than requested freely by anyone, and which reduces the risk of an unauthorised party obtaining administrative access.

The team also considered, but did not implement, a two-person approval requirement for creating a new administrator account, on the view that a single-team academic deployment does not yet need this additional control, though it is the kind of control a production deployment of the platform would likely require.

Module 3: Customer Management Customer Management maintains customer records and gives administrators a complete view of a customer. Customers maintain their own profile, while administrators use the customer list and the Customer 360 view, which brings together subscriptions, invoices, payments, loans and related records in a single read. The customer entity is linked to a company, and customers can read and write only their own data. The module is implemented for the implemented resources.

The Customer 360 view was one of the more involved pieces of the administrator experience to implement, because it draws together data from several tables that would otherwise require a customer or an administrator to open several separate screens. Building it as a single aggregated read, rather than as several separate calls from the frontend, kept the number of round trips to the backend low.

The customer record deliberately stores only the information needed to operate the platform, such as contact details and company membership, rather than a broader profile, in keeping with the general principle that a system should hold no more personal data than its features actually require.

A possible extension noted by the team is allowing a customer to request a copy or deletion of their own data, in line with common data-protection practice; this was not implemented in the current phase but is consistent with the broader principle of minimal data collection already applied to the customer record.

Module 4: Authentication and Session Management Authentication and Session Management verifies identity and maintains sessions. Users sign in with credentials, which are compared with stored password hashes, and a session is established with a token that carries the identity, role and company. Protected routes validate the token through dependencies before handlers run. The related entities are the administrators, customers, OTP verification records and session structures. The module is partially implemented: the JWT and session infrastructure exists, but OTP delivery, Google OAuth and real email transport are not connected, and development or stubbed authentication is still used in places.

The team chose to build the token and session machinery before connecting any real external identity provider, so that the rest of the backend could depend on a stable authentication interface while the OTP, email and Google

OAuth integrations were still being planned. This ordering is visible in the current status of the module, where the core mechanism is implemented but several of the delivery channels around it are not yet connected.

Sessions are represented explicitly as records rather than relying purely on the token's own expiry, which allows a future enhancement, such as an administrator being able to revoke a specific session, to be added without a change to the overall authentication design.

Multi-factor authentication beyond the currently stubbed OTP flow, such as an authenticator application, was discussed by the team as a further hardening step once real OTP delivery is connected, but was not designed in detail since it depends on decisions about the real provider that have not yet been made.

Module 5: Subscription Plans Subscription Plans lets a company define the plans it sells. Administrators create and edit plans through validated write endpoints, and customers browse the plans of their own company. The plan entity is owned by a company, and writes require the administrator role. Because plans are scoped by company, a customer of one company cannot see or subscribe to a plan of another. The module is implemented.

Plans are intentionally kept independent of the subscriptions that reference them, so that an administrator can update the description or benefits of a plan without altering the historical record of subscriptions already created against an earlier version. This separation mirrors the distinction the team drew elsewhere between mutable configuration and immutable transactional history.

A plan that has active subscriptions against it is not deleted outright in the current design; instead, administrators can withdraw it from new sign-ups, which protects the historical integrity of existing subscriptions while still letting the company retire an outdated offering.

A tiered or promotional pricing capability, where a plan's price could vary by an introductory period, was discussed as a possible enhancement but was not built, since it would have added complexity to invoice generation that was not necessary to demonstrate the core billing flow.

Module 6: Plan Benefits Plan Benefits describes the benefits included with a plan. Benefits are stored as separate records attached to a plan, which keeps plan records compact and allows benefits to be listed, added or removed independently. They are managed together with plans by administrators and read by customers, under the same company scoping. The module is implemented.

Keeping benefits as separate rows, rather than as a single text field on the plan, allows the interface to render them as a list and allows a future version of the platform to compare benefits across plans programmatically, for example to highlight what a customer would gain by upgrading, without requiring any change to the underlying schema.

Because benefits are simple structured records rather than free text, the interface can render them uniformly with icons or short labels, and a future enhancement could allow customers to compare the benefits of two plans side by side without any change to how benefits are stored.

Associating a monetary value with each benefit, so that a report could show the total benefit value included in a plan, was considered but left for a future phase, since the current benefit records are descriptive rather than valued.

Module 7: Subscription Management Subscription Management links customers to plans over time. A customer selects a plan and the backend confirms that it belongs to the customer's company before creating a subscription, which in turn triggers the generation of an invoice. Customers view their active subscriptions and administrators view subscriptions in their company. The subscription entity references the customer and the plan and is the pivot of the billing flow. The module is implemented.

A subscription was deliberately modelled as a first-class record with its own identifier, rather than as a foreign key pair on the invoice, so that a customer's subscription history remains visible even if a particular plan is later withdrawn from sale. This decision proved useful when testing the invoice generation logic, since it allowed the team to construct fixed test subscriptions independently of the current plan catalogue.

The status of a subscription, for example whether it is active or has lapsed, is derived from its associated invoices rather than stored as an independent flag that could drift out of sync, which keeps the subscription's displayed state consistent with the billing records that justify it.

Automatic renewal of a subscription at the end of a billing period is a natural next step once real payment integration exists; in the current implementation each subscription's invoicing is tied to its creation rather than to a recurring schedule.

Module 8: Invoice Management Invoice Management bills customers for their subscriptions. Invoice and invoice item records are generated by the backend from plan pricing and linked to the subscription, customer and company. Customers see only their own invoices and administrators see the invoices of their company. Generating invoices in the backend rather than in the interface ensures that the amount always follows the plan. The module is implemented.

The invoice items table exists so that a single invoice can, in future, represent more than one billable line, even though the current billing logic typically produces one item per invoice. Building this flexibility in from the start avoided a schema migration later, at the cost of a small amount of additional complexity in the current invoice generation code.

Invoice numbers are generated sequentially within a company, which gives administrators a familiar, auditable numbering scheme similar to that used in conventional accounting practice, while still keeping every invoice's number unique only within its own company's scope.

Support for adjusting an invoice after it has been generated, for instance to apply a manual discount, was intentionally left out of the current phase, since it would require careful handling to avoid weakening the guarantee that an invoice amount always follows the plan.

Module 9: Payment Management Payment Management records the settlement of invoices. A customer selects an unpaid invoice and submits a payment; the backend validates that the invoice belongs to the customer and is payable, records the payment, updates the invoice status and writes a transaction entry. Payment processing is simulated within the platform, and no real external payment gateway is connected. The module is implemented as a simulation.

Because payments are simulated, the module was designed so that the point at which a real payment gateway would be introduced is narrow and well defined: the function that currently marks a payment as successful is the function that would be replaced by a callback handler from an external provider, without requiring changes to the invoice or transaction logic around it.

Because a payment is linked to exactly one invoice, splitting a single payment across multiple invoices is not currently supported; a customer who wishes to settle several invoices must do so as separate payments, which was judged an acceptable simplification given the platform's current scale.

Support for partial payments against a single invoice, where a customer pays less than the full amount over more than one transaction, is a natural extension of the current all-or-nothing payment model and is recorded as a possible future enhancement.

Module 10: Refund Management Refund Management handles refund requests and decisions. A refund is linked to the payment it reverses and passes through request and decision states. Customers or administrators raise requests, and an administrator decides them; approved refunds produce a transaction entry. Decision endpoints require the administrator role and are scoped by company. The module is implemented.

Refunds were modelled as a separate entity from payments, rather than as a negative payment, so that the original payment record remains unchanged and auditable, and so that a payment can be partially refunded in principle without losing the record of what was originally paid.

A refund cannot exceed the amount of the payment it reverses, which the backend checks before accepting a refund decision, preventing an administrator error from producing a refund transaction larger than the original payment.

A future version could allow a refund to be issued directly to a different payment method than the one originally used, but the current implementation keeps refunds tied to the same payment record for simplicity and traceability.

Module 11: Transaction Management Transaction Management keeps a record of financial movements. Transaction entries are created by the payment, refund, disbursement and repayment logic and are written only by backend code, never directly by clients. They are read through customer statements and reports. The module is implemented.

The transaction table was added once the team recognised that payments, refunds, disbursements and repayments each needed to appear together in a customer's statement, and that deriving a statement by querying four separate

tables and merging the results in application code would be slower and more error-prone than reading a single, already-ordered transaction log.

Transactions are never edited once written; a correction, such as a reversed repayment, is represented by a new adjusting transaction rather than by modifying the original entry, preserving a complete and unaltered history of every financial movement.

A transaction-level export, similar to the CSV export already available for reports, was considered but not implemented in the current phase, since the reporting module already provides an aggregate view of the same underlying data.

Module 12: Loan Products Loan Products defines the lending products offered by a company. Administrators configure products, and customers browse them when applying for a loan. Products are owned by a company and writes require the administrator role. The module is implemented.

Loan products are configured independently of any single loan, which allows an administrator to adjust the terms offered to new applicants without altering the terms already agreed for existing loans, since those terms are copied into the loan and its terms record at the point of approval.

Interest rates and discount rules on a loan product can be updated by an administrator at any time, but such an update affects only future applications, since approved loans carry their own copy of the terms in effect at the time they were approved.

Support for tiered interest rates that vary by the size of the loan amount was discussed but not implemented, since the current loan products use a single configured rate, which was sufficient to demonstrate the schedule and repayment logic.

Module 13: Loan Applications Loan Applications captures customer requests for loans. A customer selects a product and enters an amount and tenure, and the backend validates these values against the product before creating an application with a submitted status. A customer can act only on their own applications. The module is implemented.

The application stage was kept deliberately lightweight, capturing only the amount, tenure and product, so that a customer can express interest in a loan quickly. The heavier validation, such as checking the amount against the product's limits in detail, was placed in the review stage where an administrator has visibility of the customer's full history through Customer 360.

An application that is not acted upon by an administrator within a reasonable period remains visible in the review queue indefinitely in the current implementation; an automatic expiry of stale applications is noted as a possible future enhancement rather than a feature of the present system.

Allowing a customer to attach supporting documents directly to a loan application, rather than submitting them separately through the documents module, was considered as a future usability improvement once real file storage is implemented.

Module 14: Admin Loan Review Admin Loan Review allows administrators to assess applications. A company-scoped queue lists applications, and the administrator can consult the Customer 360 view before deciding. Access requires the administrator role and is limited to the administrator's company. The module is implemented.

The review queue was implemented as a simple company-scoped list rather than as a more elaborate workflow engine, because the team judged that a small platform with a small number of administrators did not yet need features such as assignment of applications to specific reviewers, though this is noted as a possible future enhancement.

The review queue can be filtered by application status, which allows an administrator managing a large number of applications to focus on those still awaiting a decision rather than scrolling through applications that have already been approved or declined.

A future enhancement could allow more than one administrator to be assigned to review a given application, with the platform recording who ultimately made the decision, which would suit a larger administrative team than the platform currently assumes.

Module 15: Loan Approval Loan Approval converts an approved application into a loan. The backend creates the loan and its terms, records the total interest payable and the applied interest discount percentage in the columns `total_interest_payable` and `interest_discount_applied_pct`, and leaves the loan awaiting schedule generation and disbursement. State-transition checks prevent an application from being approved twice. The module is implemented.

Recording the total interest payable and the applied interest discount at the point of approval, rather than recomputing them later from the schedule, means that the terms a customer agreed to remain fixed even if the finance module's calculation logic is refined in a future version of the platform.

An application that is declined rather than approved is retained in the system with its declined status, rather than deleted, so that the customer and the administrator both retain a record of the request and its outcome.

A staged approval process, where a smaller loan can be approved by a single administrator but a larger loan requires a second administrator's confirmation, was discussed as a future control but was not built in the current phase.

Module 16: Repayment Schedule Generation Repayment Schedule Generation produces the instalment plan of a loan. The finance module reads the principal, annual rate and tenure, computes the instalment amount, divides each instalment into principal and interest, assigns due dates and stores the schedule. Inputs are validated before

calculation. The module is implemented, but the finance engine is an academic implementation and is not claimed to be production-grade.

The schedule generator was tested against the property that the sum of the principal components of every instalment must equal the loan principal, and that the final instalment must reduce the outstanding balance to exactly zero. These two properties were treated as the minimum bar for correctness before any other feature of the loan lifecycle was built on top of the schedule.

The schedule is generated once, immediately after approval, rather than being recalculated on every request, which keeps the instalment amounts stable for the customer even if the underlying interest-rate configuration on the loan product changes later.

Support for a customer requesting an alternative schedule, for example fewer, larger instalments, was considered but not implemented, since the current schedule generator assumes the tenure agreed at approval remains fixed for the life of the loan.

Module 17: Loan Disbursement Loan Disbursement releases the loan amount. An administrator initiates disbursement, and the backend checks that the loan is approved and that a schedule exists before recording the disbursement, activating the loan and writing a transaction entry. The customer is notified. The module is implemented with the same qualification as the finance engine.

Disbursement was kept as a distinct step from approval, rather than combined with it, because in a real institution the two are typically separated in time and may be performed by different staff members; keeping them separate in the data model reflects this real-world separation of duties even though the current interface allows the same administrator to perform both.

The disbursement amount recorded by the platform is always the approved principal of the loan; the platform does not currently support partial or staged disbursement, which was judged an acceptable simplification for the loan products currently modelled.

Disbursement to more than one destination, for example splitting a loan amount between two accounts, was discussed but judged unnecessary for the loan products currently modelled by the platform.

Module 18: Repayment Processing Repayment Processing applies customer payments to a loan. The amount is validated and allocated across penalty, interest and principal according to the finance module, the schedule and balances are updated, and the repayment and a transaction entry are stored. When the outstanding principal reaches zero the loan is closed. Customers can repay only their own loans. The module is implemented with the same qualification.

The order in which a repayment is allocated, penalty first, then interest, then principal, was chosen to match common lending practice, in which unpaid penalties and interest are cleared before any amount is applied to

reduce the principal. This ordering is implemented once in the finance module so that every repayment, regardless of which screen initiated it, is allocated consistently.

A repayment can be for less than a full instalment, in which case it is applied according to the same penalty-interest-principal ordering and the schedule reflects a partially paid instalment, rather than requiring repayments to exactly match a scheduled amount.

Accepting a repayment ahead of its scheduled due date, and adjusting subsequent interest accordingly, is supported by the current allocation logic in principle, though the team has not yet written a dedicated test case for early repayment and records this as an item to verify.

Module 19: Overdue Management Overdue Management marks late instalments. An administrator triggers overdue processing, and the routine selects unpaid instalments past their due date, marks them overdue, computes the overdue principal and the number of overdue instalments, and updates the loan status. The process is administrator-triggered and company-scoped. The module is implemented.

Overdue processing is currently triggered manually by an administrator rather than run automatically on a schedule, which was a deliberate simplification for the current phase of the project; a scheduled job that runs the same routine automatically is a natural extension once the platform is deployed continuously rather than run on demand.

Once an instalment is marked overdue, it remains flagged as overdue in the historical record even after it is eventually repaid, so that the overdue aging report can still reflect that a particular instalment was once late, which the team judged useful for assessing a customer's repayment reliability over time.

An automatically scheduled overdue check, rather than one triggered manually by an administrator, is recorded as a near-term enhancement, since it would make the overdue aging report reliably current without requiring a person to remember to run it.

Module 20: Repayment Reversal Repayment Reversal undoes an erroneous repayment. An administrator selects a repayment, the backend checks that it is reversible, restores instalment and balance figures, marks the repayment as reversed and records an adjusting transaction. The module is implemented, again subject to the qualification on the finance engine.

Reversal was included specifically because manual data entry, even inside an administrator interface with validation, is not immune to mistakes, and the team judged that a controlled way to undo a repayment was preferable to requiring a direct database edit, which would bypass the application's validation and audit trail entirely.

A reversal does not delete the original repayment record; instead both the original repayment and the reversal are retained, linked to one another, so that the sequence of events remains visible to anyone reviewing the loan's history later.

A future enhancement could require a documented reason to be recorded whenever a repayment is reversed, which is not currently required by the platform but would strengthen the audit trail around a sensitive operation.

Module 21: Customer Dashboard Customer Dashboard gives customers an overview of their subscriptions, invoices, loans and notifications. It is built from React components fed by customer read endpoints and currently uses preview data because the frontend is in preview mode. Customers can see only their own data. The user interface is built, and live data wiring is pending.

The dashboard was designed to answer the three questions a customer is most likely to have at a glance, namely what they are subscribed to, what they currently owe, and whether any loan they hold needs attention, before offering navigation to the more detailed screens that answer each question in full.

Because the dashboard aggregates data from several modules, it was one of the last screens completed in the frontend, since it depends on the customer-facing endpoints of the subscription, invoice, loan and notification modules all being defined first.

Once live API wiring is complete, the dashboard is expected to load slightly more slowly than in preview mode, since it will depend on several backend calls completing rather than returning pre-loaded sample data, and this is expected to be an area for performance tuning once the integration work described in Chapter 6 is finished.

Module 22: Admin Dashboard Admin Dashboard gives administrators an overview of queues, key figures and report entry points. It is built from React components fed by administrator and report endpoints and is likewise in preview mode. Access is limited to the administrator role. The user interface is built, and live data wiring is pending.

For administrators, the dashboard was designed around the queues that require action, namely applications awaiting review and refund requests awaiting decision, on the view that an administrator's first question on signing in is usually what needs to be done next rather than a general summary.

Similarly, the administrator dashboard depends on the reporting module for its summary figures, and was built once the eight report aggregates described in Chapter 7 were available from the backend.

A configurable dashboard, where an administrator can choose which figures are shown first, was discussed as a usability enhancement but was not built, since the current fixed layout was judged sufficient for the platform's present scale.

Module 23: Notifications Notifications informs users of events such as invoices, disbursements and review outcomes. Notification entities are linked to customers, and users read and update their own notifications. Real email transport is not enabled, so notifications are visible inside the platform only. The module is implemented within that limit.

Notifications were implemented as an internal record first, with real email delivery deferred, so that the events that should trigger a notification, such as an invoice being generated or a loan being disbursed, could be identified

and wired up throughout the backend before the question of how the notification is delivered to the customer was settled.

Notifications are retained after being read, with a read or unread flag rather than being deleted, so that a customer can scroll back through their notification history if needed rather than losing it once it has been viewed.

Once real email transport is connected, the team plans to let a user choose whether a given notification type is also sent by email, rather than defaulting every notification to an email copy, to avoid overwhelming customers with messages.

Module 24: Documents Documents tracks document records associated with customers. Customers submit document records and administrators review them, with a status recording the outcome. The domain and API exist within the implemented scope, but a real file-upload storage pipeline is not implemented, so the module currently represents document metadata. It is partially implemented.

The document module currently stores metadata and a review status rather than the file itself, which allowed the review workflow, submission, review and outcome, to be built and tested without also having to build a secure storage backend at the same time; the two concerns are separable, and the storage backend is treated as a distinct future enhancement.

Document records currently accept a description and a category chosen by the customer, such as identity proof or income proof, which the administrator uses to decide how to review the submission even though the underlying file itself is not yet stored by the platform.

When real file storage is introduced, the team intends to add validation of file type and size at the point of upload, consistent with the emphasis placed elsewhere in the platform on validating input before it reaches storage.

Module 25: Support Tickets Support Tickets provides customer support. Customers open tickets and exchange messages with administrators, and tickets and messages are stored under the company. A ticket is visible only to its owner and to the administrators of that company. The module is implemented.

Support tickets were scoped to the company rather than to an individual administrator, so that any administrator of the relevant company can pick up and respond to a ticket, which better matches how a small support team is likely to operate than a model that assigns every ticket to a single fixed handler.

A ticket can be reopened by a customer after being marked resolved, which the team included after recognising that a resolved issue sometimes recurs, and that requiring a customer to open an entirely new ticket in that case would lose the context of the original conversation.

Categorising tickets by topic, for example billing versus loan queries, was discussed as a way to help administrators prioritise their queue, but the current implementation treats all tickets uniformly.

Module 26: Search Search finds records across the platform. A global search box accepts text, and the backend runs tenant-scoped SQL LIKE queries over loans, invoices, payments and profiles, with the searchable scope depending on the role. Input is validated. The module is implemented; full-text search is a planned enhancement.

The decision to use SQL LIKE matching rather than a dedicated search index was made because the current data volumes are small enough that a straightforward query performs adequately, and because introducing a separate search index would have added operational complexity that the team judged unnecessary at this stage of the project.

Search results are grouped by the type of record matched, for example loans, invoices and payments shown as separate groups, rather than as a single undifferentiated list, which makes a broad search term easier to scan.

Highlighting the matched portion of text within a search result, rather than only returning the matching record, was considered as a usability improvement but was not implemented in the current phase.

Module 27: Reports Reports provides administrative insight. Eight aggregates are available: lending book, billing summary, collections by month, subscriptions by plan, overdue aging, revenue by payment method, collection efficiency and loan milestones. They are computed with aggregate queries over company-scoped data and can be exported as CSV. Advanced graphical analytics are not implemented on the backend. The module is implemented.

The eight reports were chosen because each answers a question the team judged an administrator of a lending and billing platform would routinely ask, such as how much is currently overdue or how collections are trending month to month, rather than being an exhaustive list of every possible aggregate that the underlying data could support.

Each report can be viewed for a selectable date range, which allows an administrator to compare, for instance, this month's collections against the previous month's, rather than only ever seeing an all-time aggregate.

A saved-report feature, allowing an administrator to store a particular set of filters for reuse, was discussed but not implemented, since the current reports are generated fresh on each request from a small number of filter options.

Module 28: Customer Statements Customer Statements summarises a customer's activity. The backend assembles a statement from invoices, payments, loans, repayments and transactions for the requesting customer only. The module is implemented.

The statement assembly logic reuses the same transaction log that underlies the reporting module, which keeps the figures shown to a customer consistent with the figures an administrator would see in the corresponding report, rather than risking two independently written aggregations drifting apart over time.

A statement can be generated for a selected period, mirroring the reporting module's date-range capability, so that a customer can request, for example, a statement covering only the current financial quarter.

Allowing a statement to be downloaded as a PDF, in addition to being viewed on screen, was discussed as a natural companion to the existing CSV export used for reports, but has not yet been implemented for statements specifically.

Module 29: Audit/Event Tracking Audit/Event Tracking supports the traceability of actions. Every request carries an identifier forwarded by the gateway, and event tracking structures support investigation. MongoDB activity logging is planned and is not implemented, so the module is partially implemented.

Request identifiers were introduced early in the project, before the finance module existed, because the team anticipated that tracing a single customer action across the gateway and the backend would be useful during debugging as well as for future audit purposes, and this anticipation proved correct during integration testing.

The request identifier generated by the gateway is included in backend log output, which allows a specific customer-reported issue to be traced back to the exact sequence of backend operations that handled the corresponding request, even though a dedicated audit-log table has not yet been built.

Once the planned MongoDB activity logging described in Chapter 11 is in place, the team intends to correlate its entries with the request identifiers already forwarded by the gateway, so that a single identifier can be used to trace an action across both the relational audit trail and the planned event log.

6.2 Front-End Logic (UI Rendering, State Management)

The frontend is organised into pages, reusable components and a design system. Routes for the customer and administrator areas are defined with React Router, and Axios is configured as the HTTP client. Each page is a React component that renders a view from data held in component state, and shared components such as data lists, forms, cards and status badges are reused across pages, so that a change to one component is reflected everywhere it is used. The design system supplies theme values for colours, spacing and typography, and accessibility considerations such as keyboard operability and contrast are handled once in the shared components.

State is managed with React's own state and effect mechanisms. Local component state holds form values and transient interface state such as open dialogs, while data that must be shared between components, such as the signed-in user and the selected company, is held at a higher level and passed down through props or context. Data fetching is isolated in a small service layer built on Axios, so that pages call functions such as loading invoices rather than constructing requests themselves. In preview mode this service layer returns representative data, and in live mode it will call the gateway.

The frontend currently runs in preview mode. Screens are functional and navigable, but live API wiring is not yet fully switched on. Completing this integration involves replacing preview data sources with Axios calls through the gateway, adding loading and error states, and handling token storage. The interface was built to make this replacement straightforward, because the data service layer is the only place that needs to change.

Component reuse was pursued deliberately rather than incidentally. Before a new screen was implemented, the team reviewed whether an existing component, such as the data list used for invoices, could be reused for a similar

purpose, such as listing repayments, with only its data and column definitions changed. This discipline kept the size of the component library manageable even as the number of screens grew, and it meant that a visual refinement to a shared component, such as a change to how currency amounts are displayed, improved every screen that used it at once.

6.3 API Integration

The backend API is organised by audience and resource. Customer endpoints allow a signed-in customer to read and write the resources they own, and administrator endpoints expose configuration, review and reporting operations scoped to the administrator's company. FastAPI produces an OpenAPI description that the team uses for import and specification checks. Requests and responses are JSON, apart from report exports, which are returned as CSV or text. The Authorization header carries a bearer token, the tenant context is resolved from the authenticated identity, and a request identifier is forwarded on every request.

All calls from the frontend pass through the gateway. The gateway checks the path against its allowed prefixes, applies Helmet headers, CORS policy and rate limiting, and forwards the request to the backend with the Authorization, X-Tenant-ID and request identifier headers preserved. Responses are relayed back unchanged, including CSV or text bodies, and unsupported methods receive a 405 response. This arrangement means that the frontend needs to know only one address, and that protections such as throttling are applied uniformly.

The customer API covers the implemented resources: profile, plans, subscriptions, invoices, payments, refund requests, loan products, loan applications, loans, repayments, statements, notifications, documents, support and search. The administrator API covers configuration of plans, benefits and loan products, the customer list and Customer 360, loan review, approval, schedule generation and disbursement, overdue processing, repayment reversal, refund decisions, the eight reports with CSV export, and search. Errors are returned with consistent status codes, and validation failures identify the offending field.

Integration between the layers was tested with Postman against the gateway and the backend. The integration of the frontend with the live API has not yet been completed, and this is the principal outstanding integration task. It is expected to be mostly mechanical, because the API contract is defined by the Pydantic schemas and the OpenAPI description, and the frontend service layer isolates every call.

Versioning of the API was kept implicit rather than encoded in the URL path, since the project has a single frontend consumer and the team preferred to keep endpoint paths short and readable during development. Should the platform gain additional consumers in the future, for example a mobile application, an explicit versioning scheme would be introduced at that point so that older clients could continue to function while the API evolves.

6.4 Authentication and Authorisation

Authentication verifies credentials against stored password hashes and issues tokens that identify the user, the role and the company. Passwords are never stored in plain text. On success, a session is established and a JSON Web Token is returned, and the client sends the token in the Authorization header on every later request. Protected

routes use dependencies that validate the token before the handler runs, so that an invalid or missing token is refused before any business logic executes.

Authorisation is role-based. Customer routes accept customer tokens and limit the caller to their own records, while administrator routes accept only administrator tokens and limit the caller to the administrator's company. Because the company is resolved from the authenticated identity rather than from any value that the client can change, tenant scoping does not depend on the honesty of the client. Error responses avoid revealing whether a particular credential was incorrect.

Several authentication flows are still stubbed. One-time passwords are not delivered through a real service, Google OAuth is not connected to a production provider flow, and real email transport is not enabled, so development and stubbed authentication is still used in places. These features exist as development stand-ins, and Chapter 11 describes their planned replacement. Until they are replaced, the authentication of the platform should be regarded as suitable for development and demonstration only.

The security controls of the platform are distributed across the layers. The gateway contributes header hardening, origin control, rate limiting and request identifiers. The backend contributes authentication, role checks, tenant resolution and validation. The database contributes primary, foreign, unique and check constraints. Configuration secrets are supplied through environment variables and kept out of the repository. No compliance certification, such as PCI DSS, ISO 27001 or SOC 2, is claimed for the platform.

The team also considered what an attacker with a valid but low-privilege token could attempt, such as a customer token being used against an administrator endpoint, or a token from one company being presented alongside a different company's identifier in a request header. Both scenarios were exercised during integration testing, described in Chapter 8, and in both cases the request was refused because the role and the company are read from the token itself rather than trusted from any other part of the request.

CHAPTER -7 FEATURES

7.1 List of Main Features

The platform has been conceived as a unified environment that brings together every stage of a financial company's relationship with its customers, from the moment a customer subscribes to a plan through to the repayment of a loan taken years later. Each feature has been built not as an isolated function but as part of a larger whole, in which the company is always the tenant boundary, the customer is always at the centre of the record, and every financial movement is traceable back to the event that caused it.

Foremost among these is the Company and Tenant Management feature, which establishes each financial company as an isolated tenant owning its own administrators, customers, plans and transactions. Closely related is the Authentication and Session feature, which verifies identity through hashed passwords and token-based sessions, and resolves the caller's role and company on every request so that tenant isolation is enforced consistently throughout the platform.

The Subscription and Plan Management feature allows administrators to define plans and their benefits and allows customers to subscribe to the plans of their own company. Supporting it is the Invoice and Billing feature, which generates invoices from subscriptions and tracks their status, and the Payment Processing feature, which records the settlement of those invoices as a simulated internal transaction. The Refund Management feature allows a payment to be reversed through a request and decision workflow, and the Transaction Management feature keeps a unified log of every payment, refund, disbursement and repayment.

The Loan Lifecycle feature is the most elaborate on the platform, spanning loan products, applications, administrator review, approval, repayment schedule generation, disbursement, repayment processing, overdue marking and repayment reversal. A dedicated finance module performs the monetary arithmetic that underlies every stage of this lifecycle. The Reporting and Statements feature gives administrators eight aggregate reports with CSV export and gives customers a statement of their own activity, while the Search feature allows both audiences to locate loans, invoices, payments and profiles within their own company.

The Notification, Support and Document feature keeps customers informed of events, lets them raise support tickets that administrators can answer, and lets them submit document records for administrative review. Finally, the Security and Tenant Isolation feature permeates every other feature on the platform: hashed passwords, role-based authorisation, request validation, gateway protections and company-scoped data access together ensure that no user of one company can ever reach the records of another.

Collectively, these features express the project's central aim: to provide a single, coherent platform through which a financial company can manage subscriptions and lending for its customers, with every record traceable, every tenant isolated, and every monetary calculation handled by one carefully tested module rather than scattered across the system.

7.2 How Each Feature Works

The operation of the platform rests on a consistent interaction between the React frontend, the Express gateway, the FastAPI backend and the MySQL database. Every feature, though distinct in its purpose, follows the same underlying principle: the company is resolved from the authenticated user, never from a value supplied by the client, and every piece of business logic lives in the backend rather than in the interface. The following description explains how each principal feature behaves, first from the user's standpoint and then from the technical perspective that governs its execution.

1. Company and Tenant Management Feature

User Perspective:

Customers and administrators never choose their company explicitly after the first sign-in; it is fixed to the account from the moment it is created, so every later screen simply reflects the user's own company without requiring them to select it again. An administrator sees only the customers, plans, loans and reports of their own company, and a customer sees only their own subscriptions, invoices and loans.

Technical Description:

The companies table is the ownership root of the schema, and every tenant-scoped table either references it directly or references a table that does. The backend resolves the company from the authenticated user's token on every request and applies it as a filter to the corresponding query, rather than trusting any company identifier that might appear in a request body or query string. This single rule, enforced consistently across every route, is what gives the platform its tenant isolation guarantee.

2. Authentication and Session Feature

User Perspective:

A user signs in with an email and password on a screen shared in design between the customer and administrator areas, though the two lead to different destinations afterwards. Once signed in, the user remains authenticated for the lifetime of their session and is not asked to sign in again until the token expires or they explicitly sign out.

Technical Description:

Passwords are never stored in plain text; they are hashed before being persisted, and the hash is compared against the supplied password at sign-in. On success, the backend issues a JSON Web Token encoding the user's identity, role and company, signed with a secret key. Every subsequent request carries this token in its Authorization header, and a dependency function on the backend validates it before any protected route handler executes. One-time-password delivery, Google OAuth and real email transport remain development stubs at the current stage, and development or stubbed authentication is still used in places.

3. Subscription and Plan Management Feature

User Perspective:

Customers browse the plans offered by their own company, each shown with its price and the benefits attached to it, and subscribe with a single action. Administrators reach a separate configuration screen where they can create a new plan, edit its description and pricing, and attach or remove benefits, all within their own company's catalogue.

Technical Description:

Plans and benefits are stored as separate, related records so that a plan's benefits can be listed and managed independently of the plan itself. When a customer subscribes, the backend first confirms that the selected plan belongs to the customer's own company before creating the subscription record, which prevents a customer from subscribing to a plan that was never intended for their institution.

4. Invoice and Billing Feature

User Perspective:

An invoice appears in a customer's account automatically once a subscription is created, without any separate action on the customer's part. The invoice lists the amount owed and its current status, and the same information is visible to administrators across every customer of their company.

Technical Description:

Invoice generation is triggered directly by subscription creation inside the backend, and the amount charged is always derived from the plan's current pricing rather than from any value supplied by the client. Invoice item records are stored separately from the invoice itself, which allows a single invoice to represent more than one billable line should that be required in future.

5. Payment Processing Feature

User Perspective:

A customer selects an unpaid invoice and confirms payment with a single action, after which the invoice status updates immediately and a confirmation is shown. No separate payment form or external redirection is involved, since payment processing is simulated within the platform rather than routed to a real payment provider.

Technical Description:

The backend verifies that the invoice belongs to the requesting customer and is still payable before recording the payment, updating the invoice status and writing an entry to the transaction log. The function that marks a payment as successful is the single point at which a real external payment gateway would later be introduced, without requiring any change to the invoice or transaction logic around it.

6. Refund Management Feature

User Perspective:

A customer can request a refund against a payment they have made, and the request appears in the administrator's queue for a decision. Once decided, the outcome, whether approved or rejected, is reflected in the customer's payment history.

Technical Description:

A refund is modelled as a record linked to the original payment rather than as a negative payment, which keeps the original payment unchanged and auditable. An administrator's decision triggers either a refund record and a corresponding transaction entry, on approval, or a rejected status, and the backend checks that the refund amount does not exceed the original payment before accepting the decision.

7. Loan Application and Admin Review Feature

User Perspective:

A customer browses the loan products offered by their company, selects one, and submits an application specifying the desired amount and tenure. The application then waits in a review queue that only administrators of the same company can see, where it remains until a decision is recorded.

Technical Description:

The backend validates the requested amount and tenure against the limits configured on the chosen product before accepting the application. During review, the administrator can consult the Customer 360 view, an aggregated read of the customer's subscriptions, invoices, payments and prior loans, which gives context beyond the bare application record before a decision is made.

8. Loan Approval and Repayment Schedule Feature

User Perspective:

Once an administrator approves an application, the customer sees the application's status change to approved and, shortly afterwards, a repayment schedule listing every instalment and its due date. No further action is required from the customer at this stage.

Technical Description:

Approval creates the loan record together with its terms, recording the total interest payable and the interest discount percentage applied. The finance module then computes the instalment amount from the principal, annual rate and tenure using a reducing-balance formulation, divides each instalment into principal and interest components, and stores the resulting schedule, which is tested to ensure that its principal components sum to the loan principal and that the final instalment reduces the balance to zero.

9. Loan Disbursement and Repayment Processing Feature

User Perspective:

After disbursement, a customer sees the loan marked active and can begin making repayments against the schedule shown to them. Each repayment immediately updates the instalment and the customer's outstanding balance, and the loan is marked closed once every instalment has been repaid.

Technical Description:

Disbursement is only accepted once the backend confirms that the loan is approved and that a schedule already exists for it. Each repayment is validated and then allocated by the finance module across penalty, interest and principal in that order, with the schedule, the loan balance and a transaction entry all updated together so that the loan is never left in a partially updated state.

10. Overdue Management and Repayment Reversal Feature

User Perspective:

Customers with a late instalment see it marked overdue in their schedule, and administrators see the same instalments reflected in the overdue aging report. Should a repayment be entered in error, an administrator can reverse it, after which the customer's balance returns to what it was before the mistaken repayment.

Technical Description:

Overdue processing is triggered by an administrator and marks every unpaid instalment whose due date has passed, computing the overdue principal and the number of overdue instalments for the loan. Reversal checks that the repayment is still reversible before restoring the instalment and balance figures and recording an adjusting transaction, so the original repayment is never deleted but is instead linked to the reversal that corrects it.

11. Reporting and Customer Statement Feature

User Perspective:

Administrators open the Reports screen to see eight aggregate views of their company's activity, from the lending book to collection efficiency, and can export any of them as a CSV file. Customers open their own Statements screen to see a chronological summary of their subscriptions, invoices, payments, loans and repayments.

Technical Description:

Each report corresponds to its own aggregate SQL query scoped to the requesting administrator's company, and CSV export returns the same data as CSV text, which the gateway relays unchanged. The customer statement is assembled from the same underlying transaction log that the reports draw upon, which keeps the figures a customer sees consistent with the figures an administrator sees for that customer.

12. Search Feature

User Perspective:

Both customers and administrators can type into a single search box to locate a loan, an invoice, a payment or a profile, with results grouped by the kind of record matched. A customer's search is limited to their own records, while an administrator's search covers every record belonging to their company.

Technical Description:

Search is implemented as a set of SQL LIKE queries run against the relevant tables, each scoped by company before any matching occurs. Full-text search has been identified as a future enhancement, since the current approach, while adequate for the platform's present scale, does not rank results by relevance the way a dedicated search index would.

13. Notification, Support and Document Feature

User Perspective:

Customers receive in-platform notifications when an invoice is generated, a loan is disbursed or a review outcome is recorded, and can raise a support ticket that an administrator of their company will answer through an exchange of messages on the same ticket. Document records can be submitted by a customer and reviewed by an administrator, with the outcome visible once a decision is recorded.

Technical Description:

Notifications, tickets, messages and documents are all stored as company-scoped records linked to the customer concerned. Real email delivery of notifications is not yet enabled, so they are currently visible inside the platform only, and the document feature currently stores metadata and a review status rather than an uploaded file, since a secure storage pipeline has not yet been built.

14. Security and Tenant Isolation Feature

User Perspective:

From every user's perspective, security is invisible when it is working correctly: a customer or administrator simply never encounters data belonging to another company, and an invalid or expired sign-in simply results in being asked to sign in again. There is no screen or setting through which a user can reach another company's records.

Technical Description:

Security is layered across the gateway, the backend and the database. The gateway applies Helmet headers, CORS policy, rate limiting and request identifiers; the backend applies password hashing, token validation, role-based authorisation and Pydantic validation; and the database applies primary, foreign, unique and check constraints.

Tenant isolation, the most important of these properties for a banking and fintech platform, is enforced by resolving the company from the authenticated token on every single request rather than from any value the client supplies.

Taken together, these fourteen features demonstrate a consistent pattern: the user-facing interaction is kept simple and immediate, while the technical implementation underneath enforces validation, ownership and tenant scoping on every operation. This pattern, applied uniformly rather than feature by feature, is what allows the platform to be described, with confidence, as tenant-isolated throughout rather than only in the specific areas that were tested most heavily.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

The process of verifying the reliability and functional accuracy of the platform's application programming interfaces was carried out using Postman, an industry-standard API testing environment. This tool was instrumental in ensuring that the gateway and backend behaved correctly under a variety of request scenarios, and that data integrity was preserved across service interactions. The testing methodology was designed to simulate real-world usage patterns, assess API behaviour under varying conditions, and validate both expected and exceptional responses.

Postman served as the intermediary between the user-facing interface and the application logic embedded in the backend. By invoking HTTP requests directly against API endpoints, developers were able to analyse responses in isolation without dependency on the graphical user interface. This approach enabled detailed debugging of individual services while maintaining full traceability of transactions. The testing helped in defining the request variables for the base URL, the authentication token and tenant header, and it ensured that all requests were dynamically configurable and repeatable across environments such as development and testing.

A comprehensive collection of test requests was curated within Postman, representing every major functionality of the platform, including user registration, sign-in, plan browsing, subscription, invoicing, payment, loan application, review, approval, schedule generation, disbursement, repayment, overdue processing, reversal, refund, notification, support ticket and search operations. Each request was categorised according to HTTP verb usage, GET for data retrieval, POST for resource creation, PUT for updates, and DELETE for record removal where applicable, mirroring the classification typically associated with CRUD (Create, Read, Update, Delete) operations across the entire system.

For the backend validation, each endpoint was tested using both valid and invalid input data to ensure the correct enforcement of every business rule, such as the requirement that a plan belong to the caller's company or that a loan be approved before disbursement. Parameters were tested as origin, destination, and date fields were also deliberately misconfigured to verify resilience and error handling. Authentication endpoints were also tested with expired tokens to confirm that unauthorised access attempts were rejected with proper 401 (Unauthorised) codes, and rate-limiting functionality was verified by sending rapid sequential requests, confirming that excessive hits were gracefully throttled rather than causing service failure.

Before integration testing began, each layer was verified in isolation: the backend's automated tests and manual Postman requests, the gateway's forwarding behaviour tested with a stub backend, and the frontend's components tested visually against the design in Figma. Only once each layer behaved correctly on its own did the team move to testing the layers together, on the view that defects are easier to locate when the number of moving parts under test at any one time is kept small.

The finance math test batch under pytest specifically covers the arithmetic described in Chapter 7: computing the instalment amount for a given principal, rate and tenure, verifying that the sum of principal components across a schedule equals the loan principal, checking that the final instalment reduces the outstanding balance to zero, and

verifying that a repayment is allocated across penalty, interest and principal in the intended order. These cases were chosen because they correspond to the properties that would first reveal an arithmetic defect if one were present.

Backend endpoints were also exercised with inputs designed to trigger validation failures rather than only inputs expected to succeed. Examples include a negative subscription or loan amount, a missing required field on an application, a tenure of zero instalments, and a malformed request body. In every such case the endpoint is expected to return a structured validation error rather than attempting to process the invalid input, and this expectation was checked manually through Postman for each of the affected endpoints.

8.2 Integration Testing

Integration testing verified that the gateway, the backend and the database worked together correctly as a whole, rather than only in isolation. Attention was given to the chain that begins with a subscription and ends with a payment, to the loan lifecycle from application to repayment, and to the refund flow that links a payment to its reversal. Each of these chains was exercised end to end, and the resulting records were inspected in MySQL Workbench to confirm that they were linked as the design intended.

Tenant isolation was a particular focus of integration testing, because a defect here would have serious consequences. Requests were made with the token of one company against the resources of another company, and in every case the request was refused or returned no result, which confirmed that the company filter applied consistently across subscriptions, invoices, payments, loans, reports and search. The gateway's header forwarding was also verified in this phase, by confirming that the request identifier, the Authorization header and the X-Tenant-ID header reached the backend unchanged.

Database-level integration was verified by attempting to insert rows that violated foreign key, unique or check constraints, and confirming that such inserts were rejected. The Alembic migration history was also verified by applying it to a fresh database and confirming that the resulting schema matched the current revision, 563b915e37ff, including the addition of the loan columns `total_interest_payable` and `interest_discount_applied_pct`. Where a test depended on a live frontend calling the gateway, the test could not be completed in the current preview-mode configuration, and this is recorded as an open item.

A particular effort was made to test the boundary conditions of the loan lifecycle: an application for an amount at exactly the product's limit, a repayment for exactly the outstanding balance, and an overdue check run on the exact due date of an instalment. These boundary cases are where off-by-one and rounding errors are most likely to appear, and exercising them deliberately gave the team more confidence in the finance module than exercising only typical, mid-range values would have done.

Gateway behaviour was tested separately from backend behaviour wherever possible, by pointing Postman directly at the gateway and observing whether a request was correctly forwarded, correctly rejected for an unsupported method or disallowed path, or correctly throttled once the configured rate limit was exceeded. This

separation made it possible to say with confidence whether a defect belonged to the gateway or to the backend when a request did not behave as expected.

8.3 Tools Used for Testing

Postman was the primary tool for manual API and gateway testing, used to compose requests with the correct headers and to inspect responses. pytest was used for automated backend tests, with particular emphasis on the finance arithmetic used for schedules and repayments. Python compile checks and import checks confirmed that the backend source was syntactically valid and importable, and the OpenAPI description generated by FastAPI was checked to confirm that the application's specification could be produced without error.

MySQL Workbench was used throughout testing to inspect the state of the database, to confirm that rows had been created, updated or rejected as expected, and to verify the current Alembic revision. Alembic itself was used both to apply the migration history and to confirm the current revision reported by the tool. No load-testing, browser-automation or security-scanning tools were used in the current phase of the project; these remain candidates for future testing work alongside the planned production deployment.

The tools listed above were chosen to keep the testing process lightweight and within reach of a student team, rather than adopting a heavier testing platform that would have required significant setup effort of its own. This choice reflects the same principle applied elsewhere in the project, namely that a smaller set of well-understood tools, used consistently, was preferred over a larger set of tools used only superficially.

8.4 Test Cases and Results

Test cases were prepared for each of the principal modules, covering unit-level checks, finance arithmetic, API behaviour, integration between layers, database constraints, gateway behaviour, authentication, tenant isolation and input validation. Each test case was documented with a clear description, its input, and the result expected from the specification. Where the team has direct evidence of execution, the outcome is stated; where a test case has been designed but not yet formally executed and recorded, this report states plainly that the result is to be verified, rather than presenting an unexecuted test as passed.

Two results are recorded on the basis of direct evidence available to the team. The finance math test batch, which exercises the schedule and repayment arithmetic under pytest, is recorded as having passed in an earlier run. The Alembic migration that added the loan interest columns is recorded as having been generated and applied successfully, bringing the database to revision 563b915e37ff. For the remaining test cases described in this chapter, namely the API tests for subscriptions, payments, loans and repayments, the integration tests for the billing chain and the loan lifecycle, the database constraint tests, the gateway tests for allowed prefixes, header forwarding, rate limiting and content handling, the authentication tests, the tenant isolation tests and the input validation tests, execution should be completed and the observed outcomes recorded before the report is finalised, with supporting evidence such as logs or screenshots attached in the appendix.

The overall testing approach was deliberately conservative in what it claims. Rather than asserting a pass rate that cannot be substantiated, the report distinguishes between capabilities that have been exercised with recorded

evidence and capabilities that have been designed and are ready to be tested. This approach is consistent with the honest status reporting that is used throughout the rest of the report, and it gives the team a clear checklist of testing work that remains before the platform could be considered ready for a wider audience.

The testing chapter, taken as a whole, reflects the project's general approach to claims: rather than asserting broad coverage, it records exactly what has been exercised, what evidence supports that, and what remains to be verified. This is consistent with the distinction between implemented, partially implemented and planned capabilities that is used throughout the rest of the report, and it gives a future team member or evaluator a precise list of what to check before relying on any particular part of the system.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deployment begins with the installation of the prerequisites: Python 3, Node.js in a current long-term-support release, MySQL Server 8, and Git. The project repository is cloned from GitHub, and its three parts, the frontend, the gateway and the backend, are set up in turn. The database is created first, since the backend requires it to be available before it will start.

For the backend, a Python virtual environment is created and activated, the dependencies listed in the backend's requirements are installed, the environment variables described in Section 9.2 are set, and the Alembic migration history is applied so that the database reaches the current revision. The backend is then started with Uvicorn. For the gateway, the Node.js dependencies are installed with npm, the address of the backend and the allowed origins are configured, and the gateway is started. For the frontend, the Node.js dependencies are installed and the Vite development server is started; the frontend currently runs in preview mode.

```
python -m venv venv
venv\Scripts\activate          (Windows)    |    source
venv/bin/activate              (Linux/macOS)
pip install -r requirements.txt
alembic upgrade head
uvicorn <application_module>:app --reload
```

```
cd gateway
npm install
node <entry_file>
```

```
cd frontend
npm install
npm run dev
```

Once all three parts are running, the frontend can be opened in a browser and exercised manually, and Postman can be used against the gateway to exercise the API directly. Because the frontend is in preview mode, screens display representative data rather than depending on every live call, and this should be kept in mind when demonstrating the deployed system.

The order in which the three parts are started matters in practice, even though each is a separate process. The database must be available before the backend starts, since the backend attempts to connect to it during start-up, and the backend should be available before the gateway begins forwarding requests to it, since an unavailable backend causes the gateway to return errors for every request. The frontend has the least strict ordering requirement, since it can be started at any time and will simply show connection errors for any live calls until the gateway is reachable.

Each of the three parts of the system, the frontend, the gateway and the backend, can in principle be started and stopped independently once the database is running, which proved useful during development: a change to the

frontend alone could be tested by restarting only the Vite development server, without needing to restart the gateway or the backend, which kept the development feedback loop short.

The team also kept a short written checklist of the steps in this section, used whenever a team member set up the project on a new machine, which reduced the time spent troubleshooting environment-specific problems and made it easier to confirm that a reported defect was a genuine bug in the code rather than a difference in local setup.

9.2 Environment Configuration

Configuration is supplied through environment variables so that secrets and environment-specific values are not stored in source code. The backend requires the database connection settings, the secret used to sign tokens, the token lifetime and the environment mode. The gateway requires the address of the backend, the list of allowed origins, the allowed path prefixes and the rate-limit window and limit. The frontend requires the address of the gateway and its preview-mode setting. The database requires its name and the credentials of a user with privileges limited to that database.

Secret values, including database credentials and the token signing secret, must not be committed to the repository, and separate values should be used for development and for any future production environment. A common setup issue is a backend that cannot connect to the database, which is usually resolved by confirming that MySQL is running, that the database exists and that the credentials are correct. Another common issue is a browser that blocks requests from the frontend, which is usually resolved by adding the frontend's origin to the gateway's allowed origins.

Configuration values were kept in separate files for each of the three applications rather than in one shared file, which mirrors the separation of the applications themselves and avoids a single configuration file becoming a dependency shared across parts of the system that are otherwise designed to be independent. Each application reads only the variables relevant to it and fails with a clear error message if a required variable is missing, rather than starting in a partially configured state.

9.3 Hosting of Frontend and Backend

At present, hosting is entirely local: the frontend development server, the gateway and the backend all run on the developer's own machine, and MySQL runs as a local server. No public hosting has been configured, and no domain name or TLS certificate has been obtained. This is appropriate for the current stage of the project, which is intended for development, demonstration and academic evaluation rather than for real customers.

The planned architecture for hosting places each component in its own container and uses Docker Compose to run them together during development, with a private network, a persistent volume for the database and environment configuration supplied to each container. For production, the plan is to target Kubernetes, with each service running as a Deployment behind a Service, external traffic entering through an Ingress, configuration held in ConfigMaps and secrets held in Secrets, and Prometheus and Grafana collecting and displaying metrics. MySQL would ideally be replaced by a managed or clustered database service for production use.

None of this planned hosting has been implemented or deployed. Production deployment would additionally require TLS termination, hardened secrets management, backup and recovery procedures for the database, centralised logging, monitoring and alerting, and the replacement of the current development stubs with real OTP, email, OAuth and payment provider integrations. The distinction between the current local hosting and this future production architecture is maintained clearly throughout the report so that no reader mistakes the plan for the present state.

Local hosting was also chosen because it keeps the feedback loop for development short: a change to the backend can be tested within seconds by restarting the local process, whereas a change deployed to a remote environment would involve a build and deployment step that would slow down the iterative development the team relied on throughout the project. This trade-off was judged appropriate for the current phase, where correctness and completeness of features mattered more than availability outside the development machine.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The project required the team to coordinate three technology stacks, a relational schema with many interdependent entities, and a financial workflow whose correctness is sensitive to small arithmetic errors. Keeping the frontend, the gateway, the backend and the database consistent while each was evolving was a continuing challenge, and it demanded regular integration discussions and agreed conventions between the members responsible for each layer.

Technical issues arose mainly at the boundaries between the layers: differences between the payload shapes expected by the frontend and those produced by the backend, the correct propagation of headers through the gateway, and environment configuration across three separate applications. Adding the loan interest columns required a new Alembic migration on a database that already contained structures used by other modules, and keeping the SQLAlchemy models and the migrations synchronised required care. Authentication was more complex than a simple sign-in because it combines tokens, sessions, roles and company context, and connecting real OTP delivery, email transport and Google OAuth would require external provider accounts and secure secret handling that the team has not yet arranged.

Monetary arithmetic required deliberate handling of rounding, allocation order and state transitions such as overdue marking and reversal, and errors in any of these would produce visible balance discrepancies. Wiring the frontend to the live API also proved more involved than expected, because it requires aligning data shapes, handling loading and error states, and managing tokens across many screens, and this work remains the largest outstanding task.

Several of these issues were only discovered once two layers were tested together rather than separately, which reinforced the value of the integration testing phase described in Chapter 8. A recurring pattern was that a change made to satisfy one layer's expectations, for example a field renamed in the backend's response, would silently break an assumption made in another layer's code, and catching this required the team to re-run integration tests after any change to a shared data shape.

The team also maintained a short running list of open issues throughout development, noting the symptom, the layer believed responsible, and the current status, which was reviewed briefly at the start of each work session. This lightweight practice, rather than a heavier issue-tracking tool, was judged sufficient for a team of three working closely together, and it made it easy to see at a glance which challenges had already been addressed and which remained open.

10.2 Solutions Applied

Integration issues between the layers were addressed by agreeing shared conventions for request and response shapes and by testing the affected endpoints repeatedly with Postman as changes were made. The gateway's behaviour, including forwarding of JSON and CSV or text responses, preservation of headers, correct status codes

such as 405, and IPv6-safe rate limiting, was implemented and verified incrementally rather than all at once, which made defects easier to locate.

Database and migration issues were addressed by generating the Alembic migration for the loan interest columns directly from the updated SQLAlchemy models and applying it to a test database before applying it more broadly, which allowed the team to confirm that the migration reached revision 563b915e37ff without disturbing existing data. Authentication challenges were addressed by building the token and session infrastructure first and treating OTP, email and Google OAuth as separate, clearly labelled stubs, so that the rest of the system could be developed and tested without waiting for external provider integrations.

Financial calculation challenges were addressed by isolating the arithmetic in a dedicated finance module and writing pytest cases that check the expected properties of a schedule, such as principal components summing to the loan principal and the outstanding balance reaching zero at the final instalment. Frontend integration challenges were addressed by isolating all data fetching behind a single service layer, so that switching from preview data to live API calls should require changes in one place rather than throughout the interface.

Not every issue encountered during development had a complete solution within the time available. Where a full solution would have required an external service that the team did not have access to, such as a production email provider, the team instead built a clearly labelled stand-in and documented the gap rather than leaving the feature unimplemented and undocumented. This approach is reflected in the limitations recorded in the next section.

10.3 Current Limitations or Known Bugs

A number of limitations are recorded honestly rather than concealed. Real OTP delivery is not connected to a delivery service, Google OAuth is not connected to a production provider flow, and real email transport is not enabled; development and stubbed authentication is still used in places. The frontend is still in preview mode, and live API wiring is incomplete. There is no real external payment gateway, and payments are simulated within the platform. Search uses SQL LIKE matching only, without full-text search. The document module lacks a complete real file-upload storage pipeline. The CompanyInvite model exists, but the full invite and activity log interface and API scope is not complete. Advanced visual analytics are not fully implemented on the backend for reporting. Kafka, MongoDB logging and a Spring Boot reporting service are outside the current scope, and production deployment has not been completed.

These limitations are treated as a checklist for future work rather than as defects to be hidden. Because the finance engine has not been validated to regulatory standards, it should not be relied upon outside an academic or demonstration setting, and because authentication depends on stubs in places, the platform should not be exposed to real user data until those stubs are replaced. No compliance certification is claimed for the platform.

Taken together, these limitations describe a system that is functionally complete for its academic scope but that would require further work, primarily the replacement of development stubs with real external integrations, before it could be exposed to real customers or real money. The team considers this an honest and realistic position for a

project of this size and duration, and the future enhancements in the next chapter describe the path from the current state to a more complete platform.

Beyond the specific limitations listed above, the team also recognises a more general limitation of the current phase of work: because no external service has yet been integrated, the platform has not been tested against the kind of unpredictable behaviour that real third-party services can exhibit, such as a slow or intermittently failing payment provider. This is an inherent limitation of testing against stubs rather than real integrations, and it is expected to surface new challenges once real integrations are introduced.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

A number of features are planned but not yet built. Real OTP and email delivery would connect the verification and notification flows to genuine delivery providers, with retry handling and delivery status tracking. Google OAuth would replace the stubbed flow with a production authentication flow that maps external identities to company-scoped users. A real payment gateway would replace the current simulation with a genuine external provider, including signed callbacks and reconciliation. Full-text search would replace the current SQL LIKE matching to improve relevance and performance. Document storage would add a real upload pipeline with validated file types, size limits and secure storage. Advanced analytics would add charts and trend analysis on top of the existing report aggregates, and mobile applications would provide native clients using the same APIs.

Each planned feature above was written with a specific gap in the current implementation in mind, rather than as a generic aspiration. Real OTP and email delivery, for example, directly replace the stubs described in Chapter 6, and full-text search directly replaces the SQL LIKE matching described in the search module. This traceability between a limitation recorded in Chapter 10 and an enhancement described here was maintained deliberately so that the roadmap is grounded in the actual state of the system rather than in a general wish list.

11.2 Possible Integrations or Optimisations

Beyond individual features, several integrations and optimisations are planned at the architectural level. Kafka event streaming would allow domain events such as a subscription being created or a loan being disbursed to be published for asynchronous consumers. MongoDB would store activity, usage and event logs to complement the audit tracking already present in MySQL. A separate Spring Boot reporting service would consume these events for heavier analytical workloads than the current aggregate queries. Multi-currency support would allow the platform to serve companies operating in different currencies, with explicit exchange-rate handling. Fraud detection would analyse payment and repayment patterns for anomalies, beginning with simple rules and later moving to statistical models. AI-based financial assistance could help customers understand their statements and select suitable plans, subject to privacy safeguards.

On the infrastructure side, the platform would move from local development to containerised deployment with Docker and Docker Compose, and then to Kubernetes for production, with autoscaling to meet variable load. Prometheus and Grafana would provide monitoring and dashboards for operational visibility, and cloud deployment would replace the current local MySQL instance with a managed or clustered database service. These enhancements are proposed as a roadmap rather than as completed work, and they would be undertaken roughly in the order given here, beginning with completing live operation of the current features before adding new integrations.

The order in which these enhancements would be undertaken was also considered. Completing the live wiring of the frontend and replacing the authentication stubs would come first, because they affect whether the platform can be used by real, unsupervised users at all. Real payment integration and full-text search would follow, since they improve the quality of an already-usable system. The event-streaming and analytical components, Kafka,

MongoDB logging and the Spring Boot reporting service, would come last, because they add capability rather than completing existing capability, and because they depend on the platform already running reliably in a deployed environment.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

This project designed and developed a multi-tenant subscription and billing platform for banking and fintech companies, submitted under the title “Subscription and Billing SaaS for Banking and Fintech” for the course DBSE & DBD (Course Code: 25CS1302E) by Group 18 of the Department of Computer Science and Engineering, K L (Deemed to be) University. The platform unifies customers, plans, subscriptions, invoices, payments, refunds, transactions, loans, repayments, notifications, support, documents, reports and search under a common company-scoped data model, and it is implemented with React, an Express gateway, FastAPI and MySQL 8.

Reaching this point required the team to make and revisit a number of design decisions over the course of the project, from the shape of the database schema to the division of responsibility between the gateway and the backend. Several of these decisions, such as keeping loan terms immutable once approved and keeping the gateway free of business logic, proved to simplify later work and are described in the relevant chapters above as design choices rather than as incidental facts about the implementation.

It is worth restating, in this concluding chapter, the specific pieces of direct evidence available to the team: the finance math test batch, which exercises the schedule and repayment arithmetic, is recorded as having passed in an earlier pytest run, and the Alembic migration adding the loan interest columns is recorded as having been generated and applied successfully, bringing the database to revision 563b915e37ff. These two pieces of evidence anchor the otherwise qualitative account of achievement given elsewhere in this report.

12.2 What Was Achieved

The team designed and built a relational schema covering a 26-domain-table scope, with keys, constraints and indexes, and evolved it through Alembic migrations up to revision 563b915e37ff. A FastAPI backend was implemented with customer and administrator APIs for the resources in scope, and a gateway was completed for the current scope with allowed prefixes, header and request-identifier forwarding, rate limiting and 405 handling. A finance module was implemented for schedules, disbursement, repayment, overdue marking and reversal, supported by finance tests that passed in an earlier run. Eight administrative report aggregates, CSV export, customer statements and tenant-scoped search were implemented, and a substantially built React interface with a design system and accessibility work was produced, currently operating in preview mode.

Equally, the project achieved an honest account of what remains to be done. The report distinguishes clearly between implemented, partially implemented and planned capabilities throughout, and Chapter 10 records the specific limitations that a reader or evaluator should be aware of. This honesty is itself an outcome of the project, because it reflects a disciplined approach to documentation that the team considers as important as the working code.

Looking back at the objectives set out in Chapter 1, the team judges that the core objectives, a multi-tenant database with enforced integrity, a validated and role-based API, a protective gateway, an implemented loan lifecycle and honest documentation of status, were substantially met. The objectives around a fully live frontend

and around real external integrations were only partially met, and this gap is recorded consistently throughout the report rather than only in this concluding chapter.

12.3 Skills Learned During Development

The project gave the team practical experience of relational modelling, constraints and migration management with MySQL, SQLAlchemy and Alembic, and of API design and validation with FastAPI and Pydantic. The gateway work developed skills in request forwarding, header propagation and transport-level security with Express, Helmet, CORS and rate limiting. The frontend work developed skills in component-based user interface development with React, Vite and a shared design system, including attention to accessibility and responsiveness.

Beyond individual technologies, the team learned the value of designing the data model before the interface, of treating tenant isolation as a design requirement from the outset rather than adding it later, of isolating monetary arithmetic in a dedicated, testable module, and of stating the status of each feature honestly so that planning and evaluation are both easier. Testing with pytest and Postman, and recording only the results that are actually supported by evidence, was itself a lesson in disciplined engineering practice. Finally, version control and team collaboration with Git and GitHub allowed three members to work on complementary areas of the same system without losing coherence, which is itself a skill that the course intended the team to develop.

Perhaps the most transferable lesson from the project is that a system's documentation is only as useful as it is accurate. A report that claims more than a system actually does misleads the very people, evaluators, future developers and the team's own future selves, who most need an accurate picture in order to decide what to trust and what to build next. The team has tried to apply that lesson consistently throughout this report, and considers it as much a part of the project's output as the source code itself.

CHAPTER -13 REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education has been listed above for its treatment of software processes; the team additionally consulted Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley, for guidance on keeping the backend's service layer and the frontend's component structure maintainable as the project grew.

Tutorials and Learning Resources

Python Software Foundation. (2026). Python Documentation. Retrieved from <https://docs.python.org/3/>

FastAPI. (2026). FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com/>

MDN Web Docs. (2026). Web Technologies Documentation. Retrieved from <https://developer.mozilla.org>

W3Schools. (2026). HTML, CSS, and JavaScript Tutorials. Retrieved from <https://www.w3schools.com>

APIs and Tools Used

SQLAlchemy. (2026). SQLAlchemy Documentation. Retrieved from <https://docs.sqlalchemy.org/>

Alembic. (2026). Alembic Documentation. Retrieved from <https://alembic.sqlalchemy.org/>

Pydantic. (2026). Pydantic Documentation. Retrieved from <https://docs.pydantic.dev/>

Oracle. (2026). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>

Meta. (2026). React Documentation. Retrieved from <https://react.dev/>

Vite. (2026). Vite Documentation. Retrieved from <https://vite.dev/>

Express. (2026). Express.js Documentation. Retrieved from <https://expressjs.com/>

Postman. (2026). Postman Learning Center. Retrieved from <https://learning.postman.com/docs/>

Figma. (2026). Figma Help Center. Retrieved from <https://help.figma.com/>

GeeksforGeeks. (2026). Database Design and Normalisation. Retrieved from <https://www.geeksforgeeks.org>

Real Python. (2026). FastAPI and SQLAlchemy Tutorials. Retrieved from <https://realpython.com>

Documentation and Research References

Docker Inc. (2026). Docker Documentation. Retrieved from <https://docs.docker.com/>

The Kubernetes Authors. (2026). Kubernetes Documentation. Retrieved from <https://kubernetes.io/docs/>

Prometheus Authors. (2026). Prometheus Documentation. Retrieved from <https://prometheus.io/docs/>

Grafana Labs. (2026). Grafana Documentation. Retrieved from <https://grafana.com/docs/>

Apache Software Foundation. (2026). Apache Kafka Documentation (planned technology). Retrieved from <https://kafka.apache.org/documentation/>

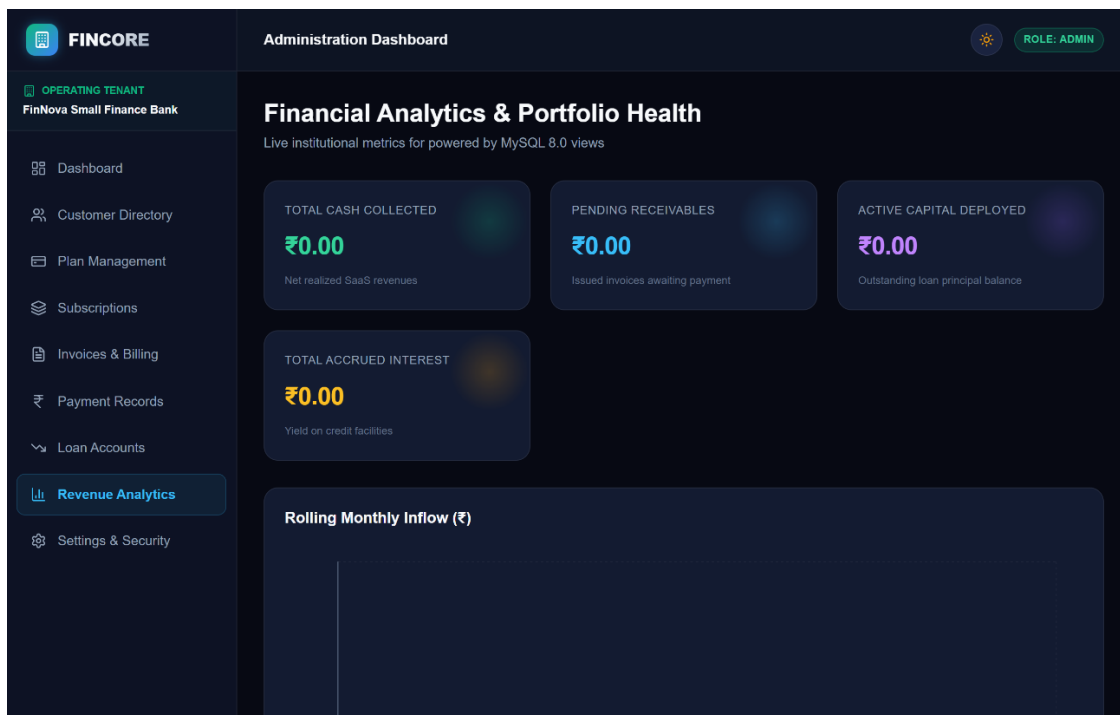
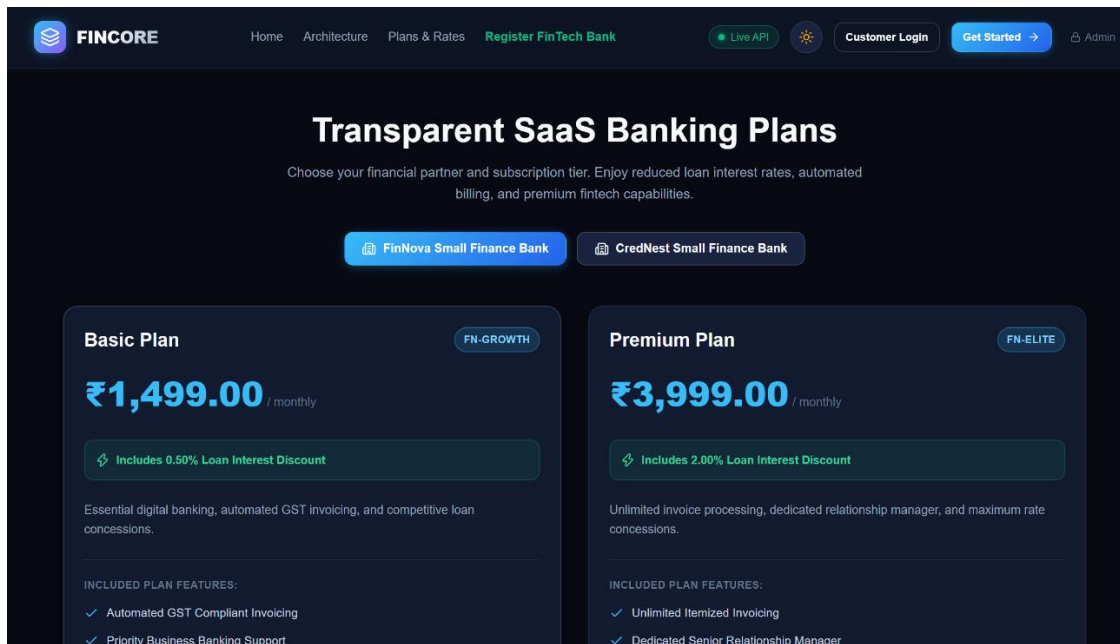
MongoDB Inc. (2026). MongoDB Documentation (planned technology). Retrieved from <https://www.mongodb.com/docs/>

Reserve Bank of India. (2026). Guidelines on Digital Lending. Retrieved from <https://www.rbi.org.in>

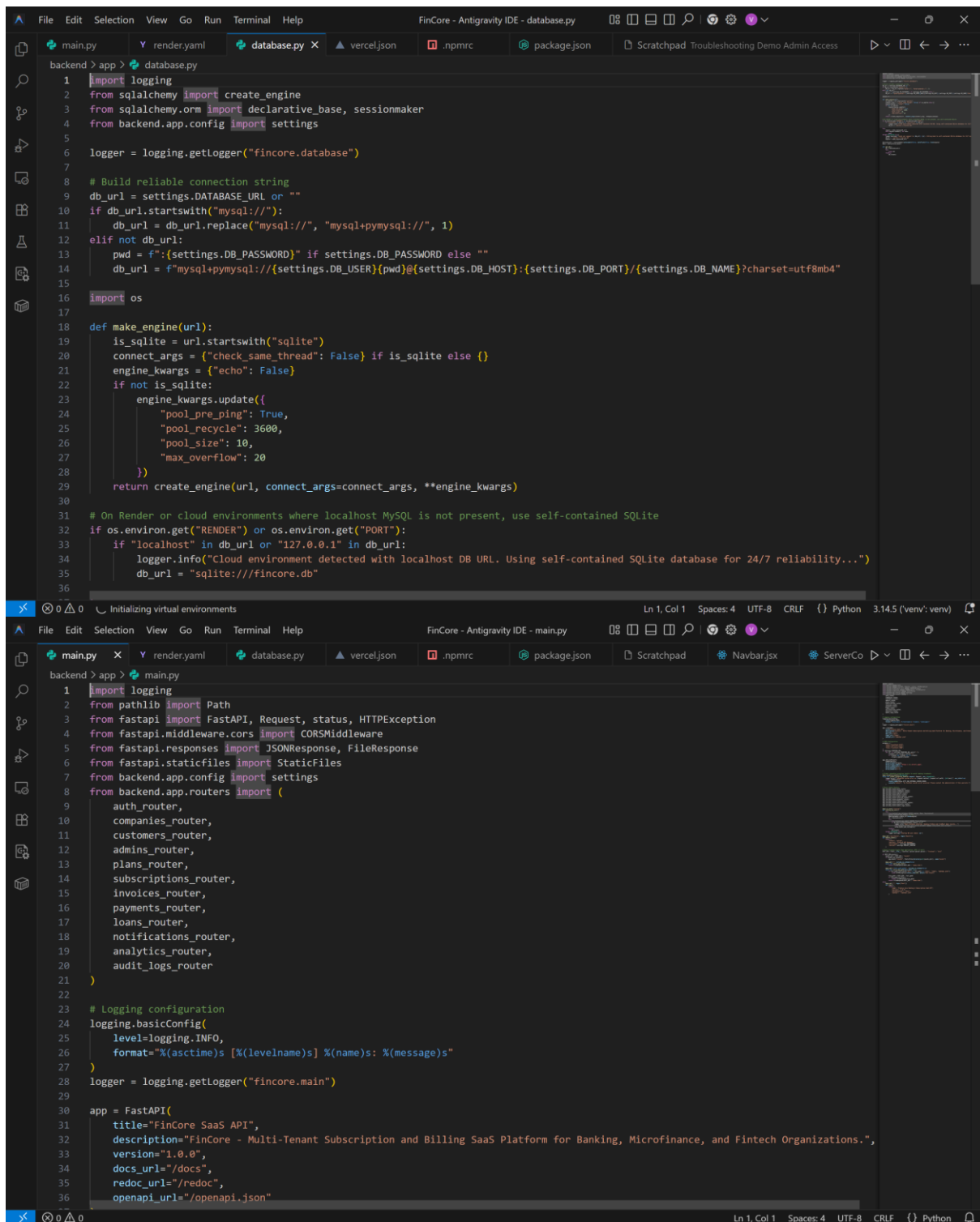
OWASP Foundation. (2026). OWASP Cheat Sheet Series. Retrieved from <https://cheatsheetseries.owasp.org>

CHAPTER -14 APPENDICES

Screenshots of the website :



Sample code snippets :



The image displays two screenshots of the FinCore - Antigravity IDE, showing code snippets for database.py and main.py.

database.py snippet:

```
1 import logging
2 from sqlalchemy import create_engine
3 from sqlalchemy.orm import declarative_base, sessionmaker
4 from backend.app.config import settings
5
6 logger = logging.getLogger("fincore.database")
7
8 # Build reliable connection string
9 db_url = settings.DATABASE_URL or ""
10 if db_url.startswith("mysql://"):
11     db_url = db_url.replace("mysql://", "mysql+pymysql://", 1)
12 elif not db_url:
13     pwd = f"{settings.DB_PASSWORD}" if settings.DB_PASSWORD else ""
14     db_url = f"mysql+pymysql://{settings.DB_USER}:{pwd}@{settings.DB_HOST}:{settings.DB_PORT}/{settings.DB_NAME}?charset=utf8mb4"
15
16 import os
17
18 def make_engine(url):
19     is_sqlite = url.startswith("sqlite")
20     connect_args = {"check_same_thread": False} if is_sqlite else {}
21     engine_kwargs = {"echo": False}
22     if not is_sqlite:
23         engine_kwargs.update({
24             "pool_pre_ping": True,
25             "pool_recycle": 3600,
26             "pool_size": 10,
27             "max_overflow": 20
28         })
29     return create_engine(url, connect_args=connect_args, **engine_kwargs)
30
31 # On Render or cloud environments where localhost MySQL is not present, use self-contained SQLite
32 if os.environ.get("RENDER") or os.environ.get("PORT"):
33     if "localhost" in db_url or "127.0.0.1" in db_url:
34         logger.info("Cloud environment detected with localhost DB URL. Using self-contained SQLite database for 24/7 reliability...")
35         db_url = "sqlite:///fincore.db"
36
```

main.py snippet:

```
1 import logging
2 from pathlib import Path
3 from fastapi import FastAPI, Request, status, HTTPException
4 from fastapi.middleware.cors import CORSMiddleware
5 from fastapi.responses import JSONResponse, FileResponse
6 from fastapi.staticfiles import StaticFiles
7 from backend.app.config import settings
8 from backend.app.routers import (
9     auth_router,
10     companies_router,
11     customers_router,
12     admins_router,
13     plans_router,
14     subscriptions_router,
15     invoices_router,
16     payments_router,
17     loans_router,
18     notifications_router,
19     analytics_router,
20     audit_logs_router
21 )
22
23 # Logging configuration
24 logging.basicConfig(
25     level=logging.INFO,
26     format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
27 )
28 logger = logging.getLogger("fincore.main")
29
30 app = FastAPI(
31     title="FinCore SaaS API",
32     description="FinCore - Multi-Tenant Subscription and Billing SaaS Platform for Banking, Microfinance, and Fintech Organizations.",
33     version="1.0.0",
34     docs_url="/docs",
35     redoc_url="/redoc",
36     openapi_url="/openapi.json"
37 )
```

Installation/setup instructions

The platform is installed in three parts, corresponding to the backend, the gateway and the frontend, together with a MySQL database. Detailed commands are given in Chapter 9. In outline: install Python, Node.js and MySQL Server 8; clone the project repository from GitHub; create the project database in MySQL and a limited-privilege user for it; configure the environment variables for the database connection, the token signing secret, the backend address, the allowed origins and the rate limits; create a Python virtual environment, install the backend dependencies and apply the Alembic migrations so that the database reaches revision 563b915e37ff; start the backend with Uvicorn; install and start the gateway; install the frontend dependencies and start the Vite development server; and finally open the frontend in a browser, bearing in mind that it currently runs in preview mode.

User manual or guide

A customer opens the application, selects the financial company, and signs in; development and stubbed authentication is used in places, so OTP and Google sign-in are not yet connected to real providers. The customer browses plans and their benefits and subscribes to a plan, which generates an invoice that can then be paid from the Invoices screen, with confirmation shown once payment is recorded. A refund can be requested on a payment if required. From the Loans screen, the customer chooses a loan product, enters an amount and tenure, and submits the application; after approval and disbursement, the repayment schedule becomes visible and instalments can be repaid. The Statements screen shows account activity, and the Notifications screen shows system messages. Support tickets can be raised and answered through message exchange, and the search box finds loans, invoices and payments belonging to the signed-in customer.

An administrator signs in to the administrator area and creates plans, benefits and loan products for their company. From the Customers screen, the administrator can open the Customer 360 view for a full picture of a customer's activity. The loan review queue lists applications awaiting a decision; once approved, the administrator generates the repayment schedule and disburses the loan. Overdue processing marks late instalments, and a mistaken repayment can be reversed. Refund requests are reviewed and decided from the Refunds screen. The Reports screen provides the eight report aggregates and CSV export, support tickets and document records are handled from their respective screens, and the search box is scoped to the administrator's own company.

INTRODUCTION

- The Subscription and Billing SaaS for Banking and Fintech is a system designed to simplify how banks and fintech companies manage their subscription-based services. It brings customers, subscription plans, billing, invoices, and payments together in one platform. Customers can choose a plan, manage their subscriptions, make payments, and view their billing history. Administrators can manage plans, customers, transactions, and revenue.
- The project also focuses on database management, backend development, and distributed systems. It uses databases such as PostgreSQL and MongoDB, along with backend APIs and microservices to make the system secure, reliable, and scalable. Overall, the project provides a practical solution for managing modern financial subscription and billing services.

Hyderabad, Telangana, India 🇮🇳
 Gcw3+wm6, Aleap Industrial Area, Gajularamaram,
 Hyderabad, Telangana 500043, India
 Lat 17.547261° Long 78.40441°
 Tuesday, 25/08/2026 09:04 AM GMT +05:30

